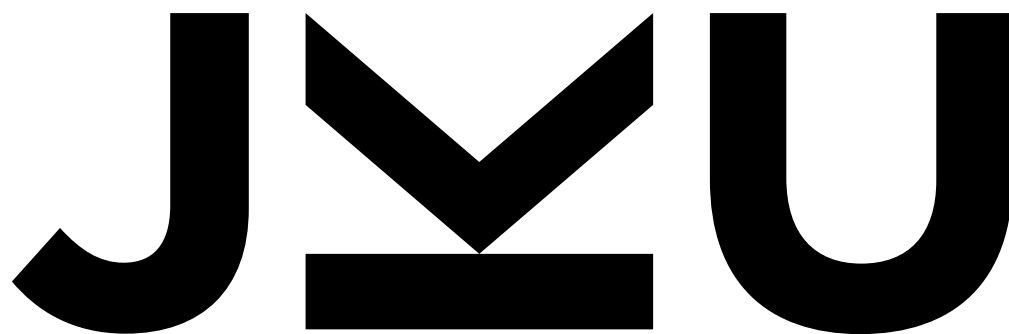
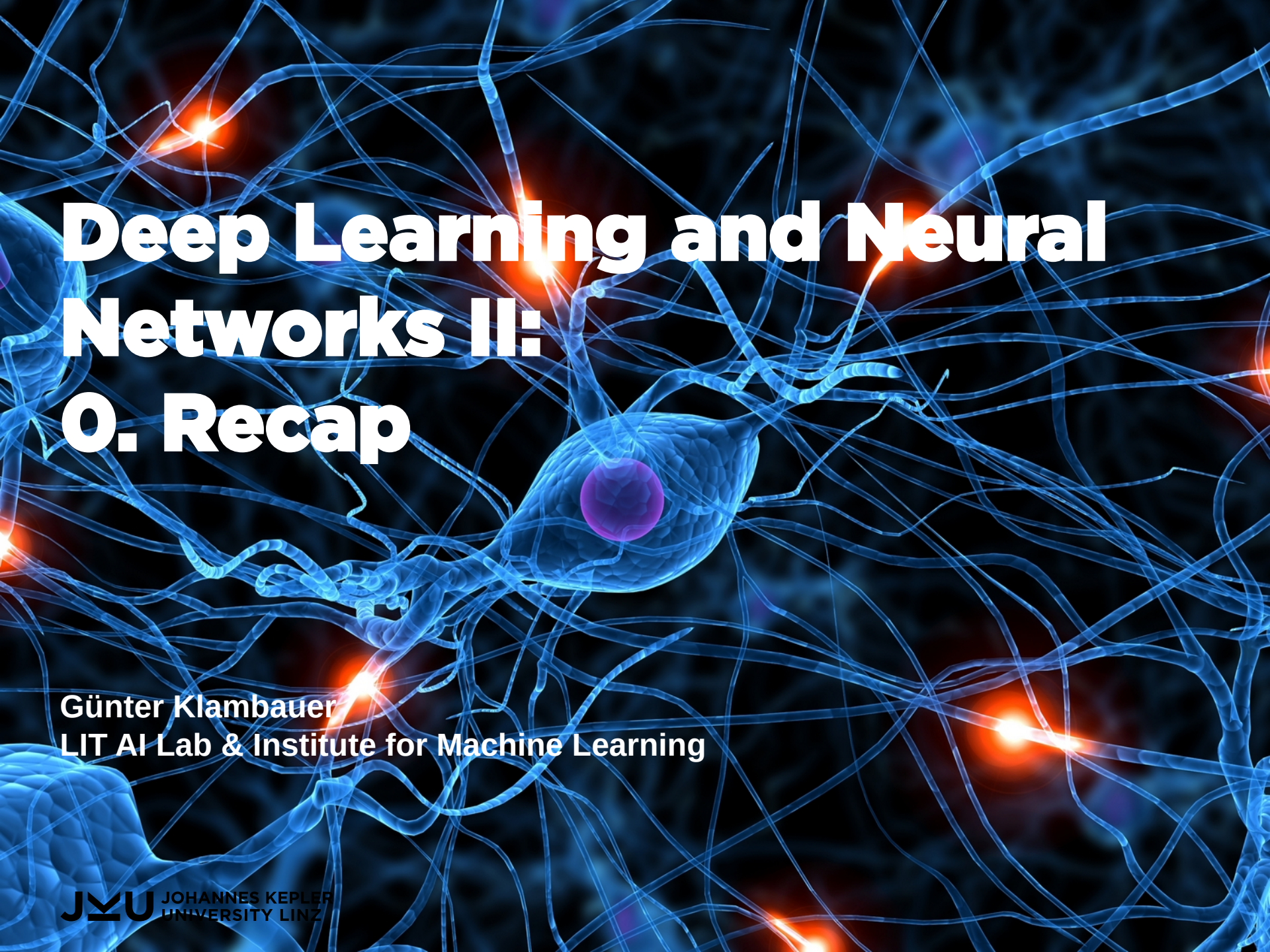




**JOHANNES KEPLER
UNIVERSITY LINZ**



Deep Learning and Neural Networks II: 0. Recap

Günter Klambauer
LIT AI Lab & Institute for Machine Learning

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Brief overview on DLNN1

4. Basic neural network architectures
 1. Linear model: Neuron
 2. Perceptron
 3. Linear neuron with sigmoid activation: Logistic regression
 4. Linear neuron with softmax: Softmax regression
 5. Single-layer networks cannot solve XOR
 6. Multi-layer perceptrons
 7. (Deep) Feed-forward neural networks
 8. Jacobian
 9. Hessian
 10. Efficient training of DNNs and MLPs: Basic tricks of the trade

Brief overview on DLNN1

5. Generalization error, test set and CV
6. Convolutional neural networks
7. Learning methods: Basic algorithms
8. Regularization
9. Initialization
10. Normalization

Structure of a deep neural network (DNN)

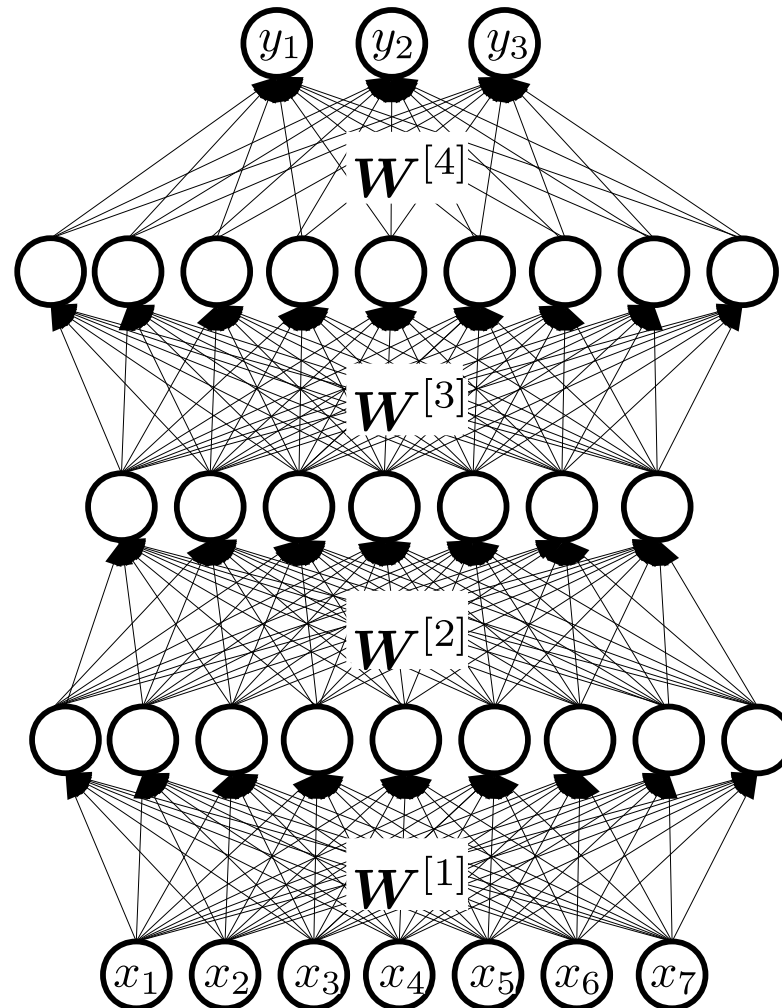
output layer

hidden layer 3

hidden layer 2

hidden layer 1

input layer



$$\hat{y} = \sigma(\underbrace{W^{[4]} a^{[3]}}_{=s^{[4]}})$$

$$a^{[3]} = f(\underbrace{W^{[3]} a^{[2]}}_{=s^{[3]}})$$

$$a^{[2]} = f(\underbrace{W^{[2]} a^{[1]}}_{=s^{[2]}})$$

$$a^{[1]} = f(\underbrace{W^{[1]} x}_{s^{[1]}})$$

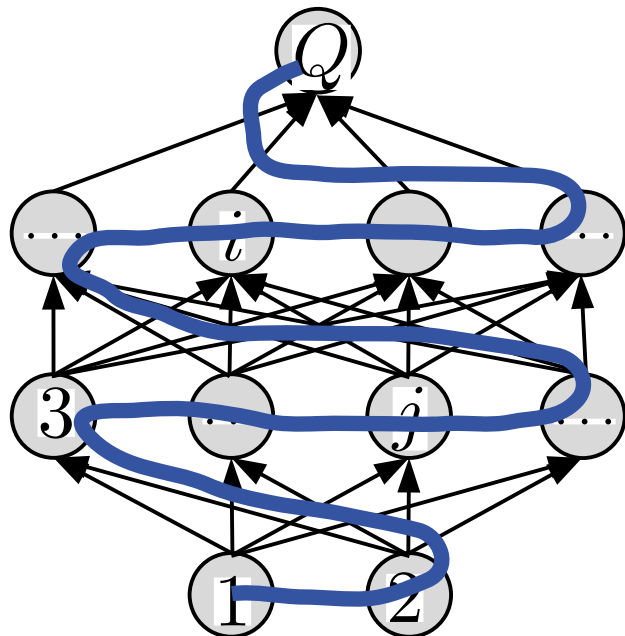
x

DNNs: notation

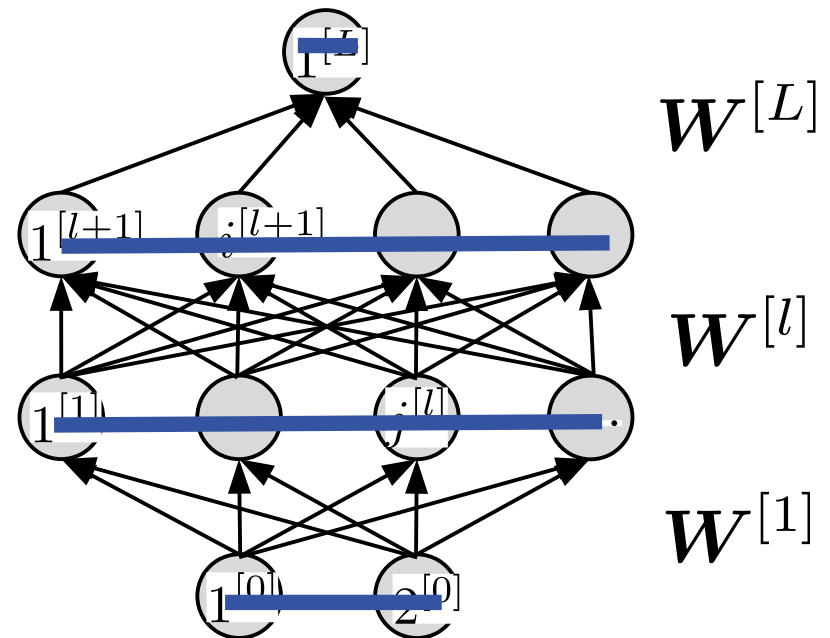
- $\mathbf{x}$: input for layer 0; equivalent to $\mathbf{a}^{[0]}$
- $\mathbf{W}^{[l]}$: weight matrix connecting layer $l - 1$ and layer l
- $\mathbf{s}^{[l]}$: pre-activations of layer l
- $\mathbf{a}^{[l]}$: activations of layer l
- f : activation function that is applied element-wise to a vector.

Remark: Notation for MLP and for deep FNNs

MLP:



FNN:



DNNs: Forward pass

- Activations from one layer to the next layer:

$$\mathbf{a}^{[l]} = f(\mathbf{s}^{[l]}) = f(\mathbf{W}^{[l]} \mathbf{a}^{[l-1]})$$

- Full function:

$$\hat{\mathbf{y}} = g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]}) = \sigma(\mathbf{W}^{[L]} (\dots f(\mathbf{W}^{[2]} f(\mathbf{W}^{[1]} \mathbf{x})) \dots))$$

- With bias units

$$\mathbf{a}^{[l]} = f(\mathbf{s}^{[l]}) = f(\mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}).$$

Why non-linearities?

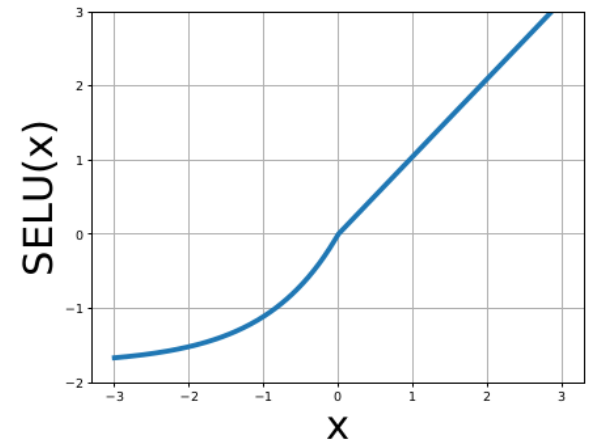
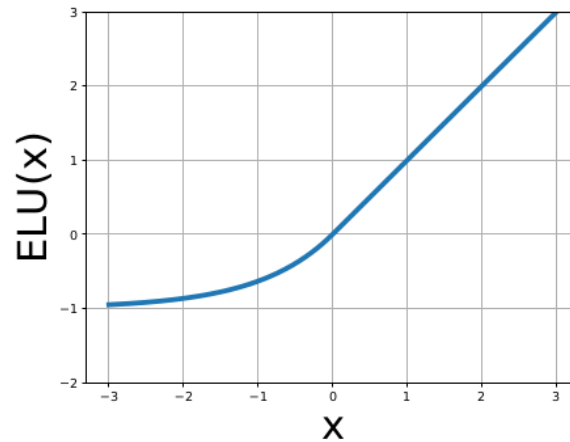
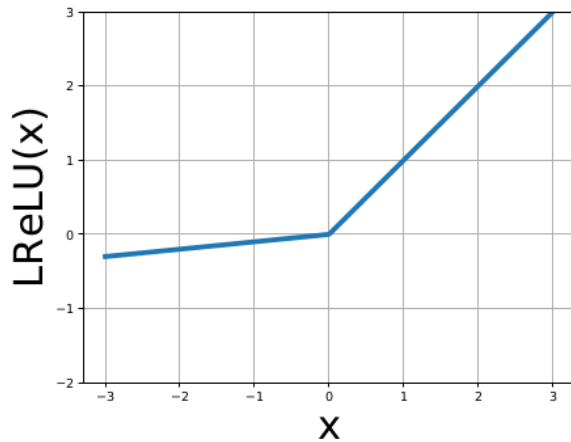
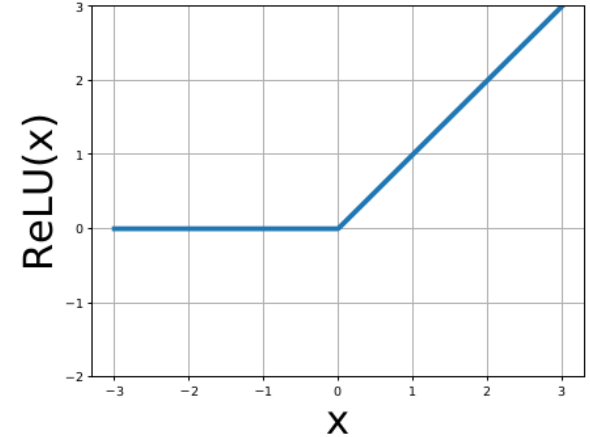
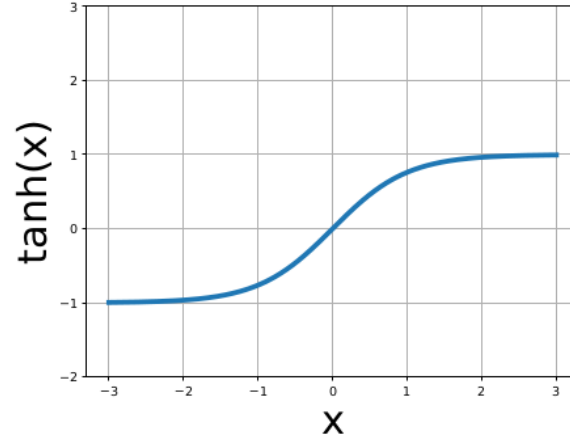
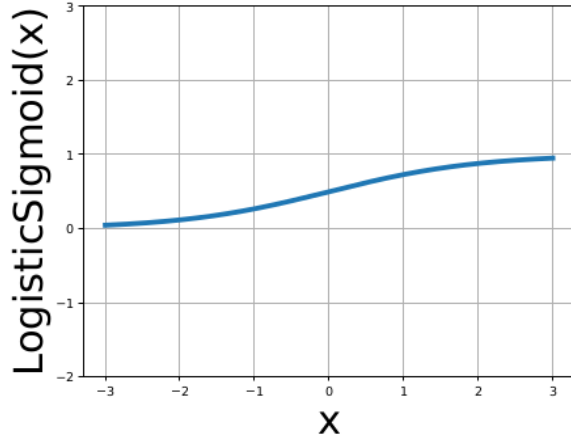
- With non-linearity:

$$g(x; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]}) = \sigma(\mathbf{W}^{[L]}(\dots f(\mathbf{W}^{[2]} f(\mathbf{W}^{[1]} x)) \dots))$$

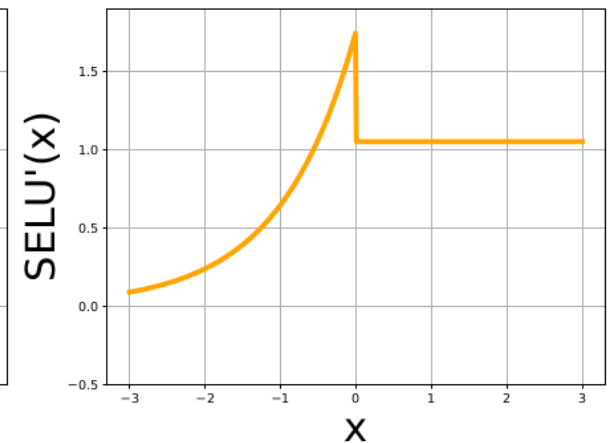
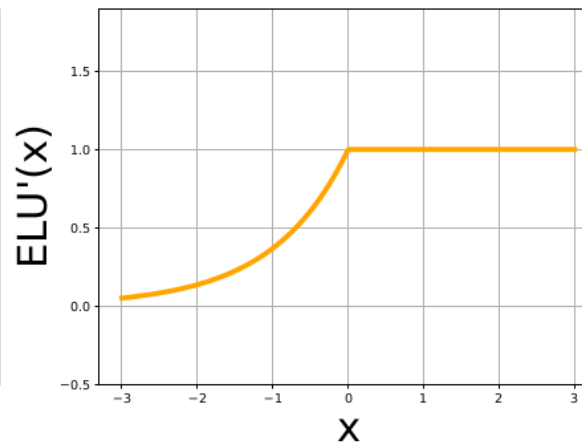
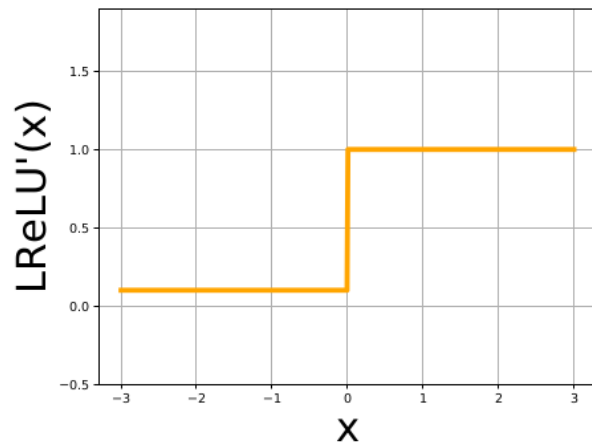
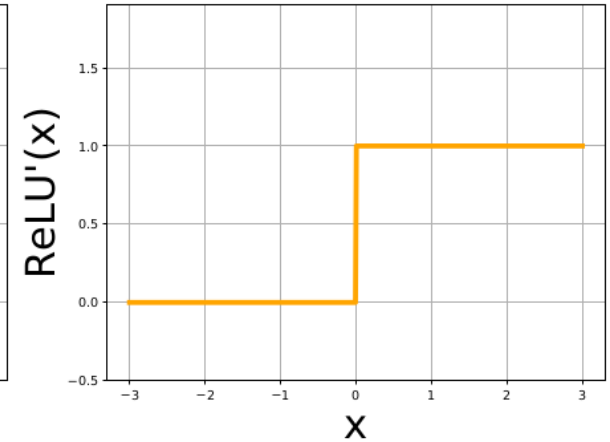
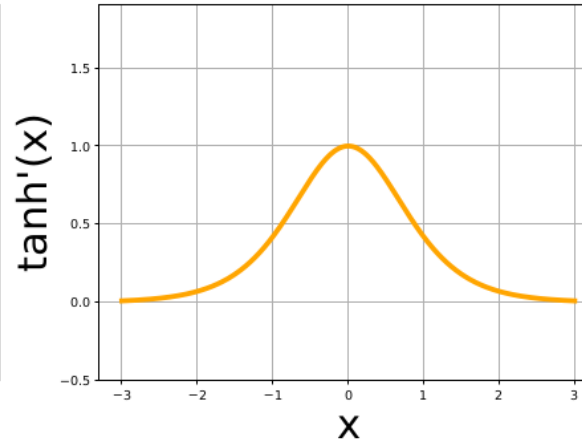
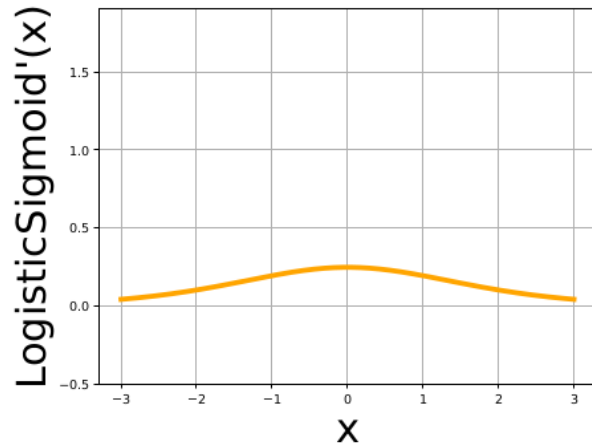
- Without non-linearity:

$$g(x; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]}) = \sigma(\underbrace{\mathbf{W}^{[L]} \dots \mathbf{W}^{[1]}}_{=\hat{\mathbf{W}}} x)$$

A note on activation functions



A note on activation functions: derivatives



DNN: Forward pass

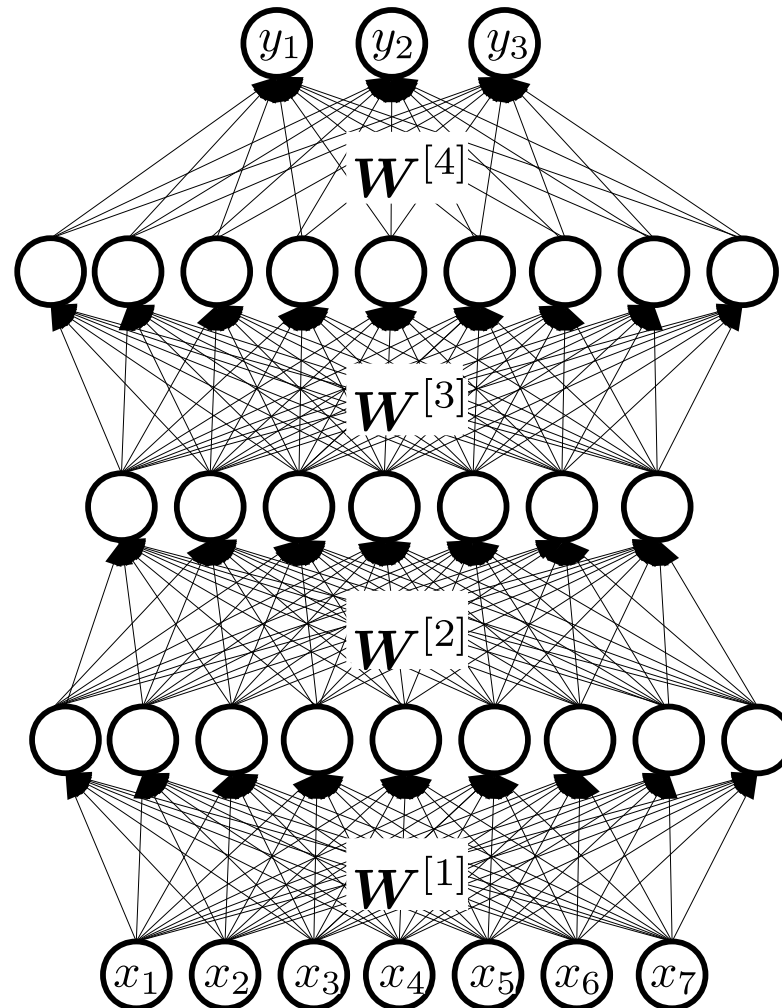
output layer

hidden layer 3

hidden layer 2

hidden layer 1

input layer



$$\hat{y} = \sigma(\underbrace{W^{[4]} a^{[3]}}_{=s^{[4]}})$$

$$a^{[3]} = f(\underbrace{W^{[3]} a^{[2]}}_{=s^{[3]}})$$

$$a^{[2]} = f(\underbrace{W^{[2]} a^{[1]}}_{=s^{[2]}})$$

$$a^{[1]} = f(\underbrace{W^{[1]} x}_{s^{[1]}})$$

x

Backpropagation in DNNs

- Without loss of generality, we derive with respect to a weight in the first layer:

$$\frac{\partial}{\partial w_{ij}^{[1]}} L(\mathbf{y}, g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]})) = \underbrace{\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}}_{=:A} \cdots \underbrace{\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}}}_{=:B} \cdots \underbrace{\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=:C}$$

- Now we treat the expressions A, B and C.

Backpropagation in DNNs

- We first define:

$$\delta^{[l]} := \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{s}^{[l]}}$$

Backpropagation in DNNs

- Without loss of generality, we derive with respect to a weight in the first layer:

$$\frac{\partial}{\partial w_{ij}^{[1]}} L(\mathbf{y}, g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]})) = \underbrace{\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}}_{=:A} \cdots \underbrace{\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}}}_{=:B} \cdots \underbrace{\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=:C}$$

- Now we treat the expressions A, B and C.

Backpropagation in DNNs

- Without loss of generality, we derive with respect to a weight in the first layer:

$$\frac{\partial}{\partial w_{ij}^{[1]}} L(\mathbf{y}, g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]})) = \underbrace{\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}}_{=:A} \cdots \underbrace{\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}}}_{=:B} \cdots \underbrace{\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=:C}$$

- A $\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}$: Depends on the choice of loss function and activation. If canonical links are chosen, this can result in

$$\delta^{[L]} = (\hat{\mathbf{y}} - \mathbf{y})^T$$

Backpropagation in DNNs

- Without loss of generality, we derive with respect to a weight in the first layer:

$$\frac{\partial}{\partial w_{ij}^{[1]}} L(\mathbf{y}, g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]})) = \underbrace{\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}}_{=:A} \cdots \underbrace{\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}}}_{=:B} \cdots \underbrace{\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=:C}$$

- $B \quad \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} :$
 $\mathbf{s}^{[l]} = \mathbf{W}^{[l]} f(\mathbf{s}^{[l-1]})$
 $\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} = \mathbf{W}^{[l]} \text{diag} \left(f'(\mathbf{s}^{[l-1]}) \right)$

Backpropagation in DNNs

- Without loss of generality, we derive with respect to a weight in the first layer:

$$\frac{\partial}{\partial w_{ij}^{[1]}} L(\mathbf{y}, g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]})) = \underbrace{\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}}_{=:A} \cdots \underbrace{\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}}}_{=:B} \cdots \underbrace{\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=:C}$$

- Recursive formula for deltas:*

$$\delta^{[l-1]} = \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{s}^{[l]}} \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} = \delta^{[l]} \underbrace{\mathbf{W}^{[l]} \text{diag} \left(f'(\mathbf{s}^{[l-1]}) \right)}_{\mathbf{J}^{[l]}}$$

Backpropagation in DNNs

- Without loss of generality, we derive with respect to a weight in the first layer:

$$\frac{\partial}{\partial w_{ij}^{[1]}} L(\mathbf{y}, g(\mathbf{x}; \mathbf{W}^{[1]}, \dots, \mathbf{W}^{[L]})) = \underbrace{\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{s}^{[L]}}}_{=:A} \cdots \underbrace{\frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}}}_{=:B} \cdots \underbrace{\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}}}_{=:C}$$

- C:** $\frac{\partial \mathbf{s}^{[1]}}{\partial w_{ij}^{[1]}} = \frac{\partial}{\partial w_{ij}^{[1]}} \mathbf{W}^{[1]} \mathbf{a}^{[0]} = \begin{pmatrix} 0 \\ \vdots \\ a_j^{[0]} \\ \vdots \\ 0 \end{pmatrix},$

where the non-zero entry $a_j^{[0]}$ is at the i -th position.

Backpropagation in DNNs

- Overall we find

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{[l]}} = \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{s}^{[l]}} \frac{\partial \mathbf{s}^{[l]}}{\partial w_{ij}^{[l]}} = (\boldsymbol{\delta}^{[l]})_i (\mathbf{a}^{[l-1]})_j$$

or conveniently:

$$\frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{[l]}} = \boldsymbol{\delta}^{[l]} \mathbf{a}^{[l-1]T}.$$

Summary of backprop

- Calculate deltas at output units $\delta^{[L]}$
- Backpropagate deltas through network using

$$\delta^{[l-1]} = \frac{\partial L(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{s}^{[l]}} \frac{\partial \mathbf{s}^{[l]}}{\partial \mathbf{s}^{[l-1]}} = \delta^{[l]} \underbrace{\mathbf{W}^{[l]} \text{diag} \left(f'(\mathbf{s}^{[l-1]}) \right)}_{\mathbf{J}^{[l]}}$$

- Calculate weight update

$$\Delta \mathbf{W}^{[l]} = -\eta \delta^{[l]} (\mathbf{a}^{[l-1]})^T$$

The vanishing gradient problem

- We know from above:

$$\delta^{[l-1]} = \delta^{[l]} \underbrace{\mathbf{W}^{[l]} \text{diag} \left(f'(\mathbf{s}^{[l-1]}) \right)}_{\mathbf{J}^{[l]}} = \delta^{[l]} \mathbf{J}^{[l]}$$

- Thus we have: $\|\mathbf{J}^{[l]}\| \leq k \leq 1$

$$\|\delta^{[1]}\| = \|\delta^{[L]}\| \prod_{l=L}^2 \|\mathbf{J}^{[l]}\| \leq k^{L-1} \|\delta^{[L]}\| \Rightarrow \|\delta^{[1]}\| \approx 0$$

The vanishing gradient problem

output layer

$$\delta^{[L]}$$

hidden layer 3

$$\delta^{[L-1]} = \delta^{[L]} J^{[L]}$$

hidden layer 2

$$\delta^{[L-2]} = \delta^{[L-1]} J^{[L-1]}$$

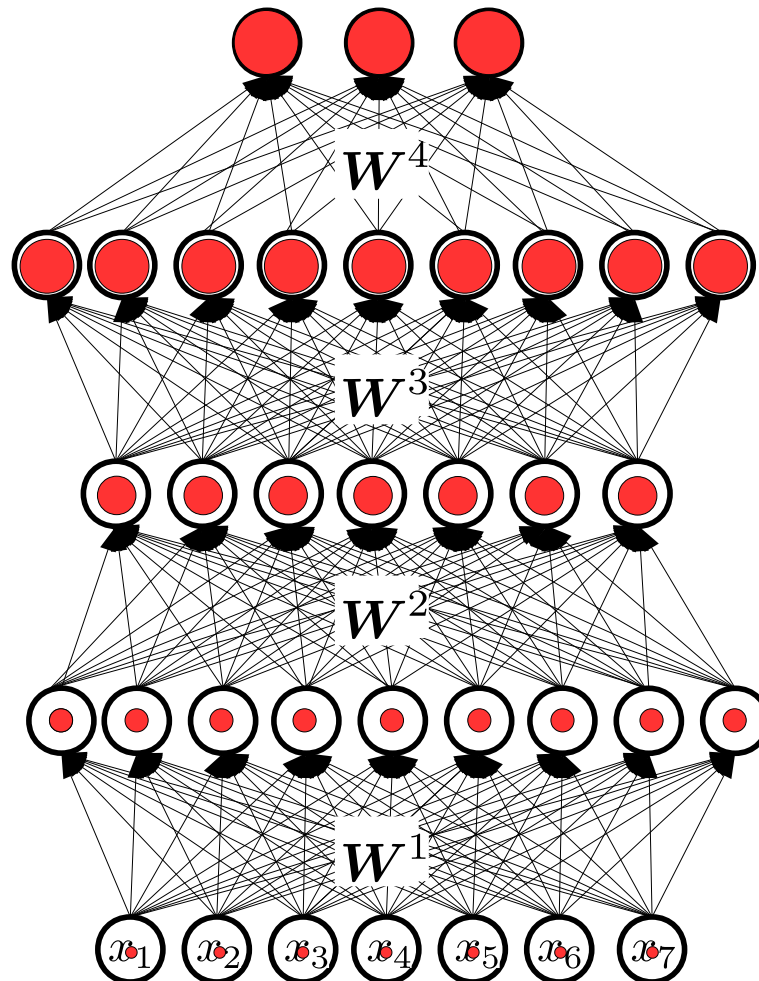
hidden layer 1

$$\delta^{[L-3]} = \delta^{[L-2]} J^{[L-2]}$$

$\vdots$

input layer

$$\delta^{[0]} = \delta^{[1]} J^{[1]}$$



Deep feed-forward neural networks

- Same principle as MLPs
 - Layers of interconnected neurons
 - More layers; more neurons
 - Different activation functions

Table 1: Comparison of typical multi-layer perceptrons and deep feed-forward neural networks (informal guidelines).

	MLP	DNN
Number of hidden layers	1 or few	> 1 up to several hundreds
Number of neurons per layer	< 10	> 100
Activation function	tanh, sigmoid	ReLU, SELU

Ameliorations of the VGP for feed-forward NN: Outlook

- Rectified linear units
- Initialization strategies
- Skip connections (ResNets)
- ... (see later)

Online, stochastic and batch training

- Online learning/training: single sample from training set is propagated through network and then a parameter update is performed.

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} R_{\text{emp}}(\mathbf{y}, \mathbf{x}, \mathbf{w})|_{\mathbf{w}^{\text{old}}}$$

- Full-batch learning/training: all samples from training set are propagated through network and then a parameter update is performed.

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} R_{\text{emp}}(\mathbf{y}, \mathbf{X}, \mathbf{w})|_{\mathbf{w}^{\text{old}}}$$

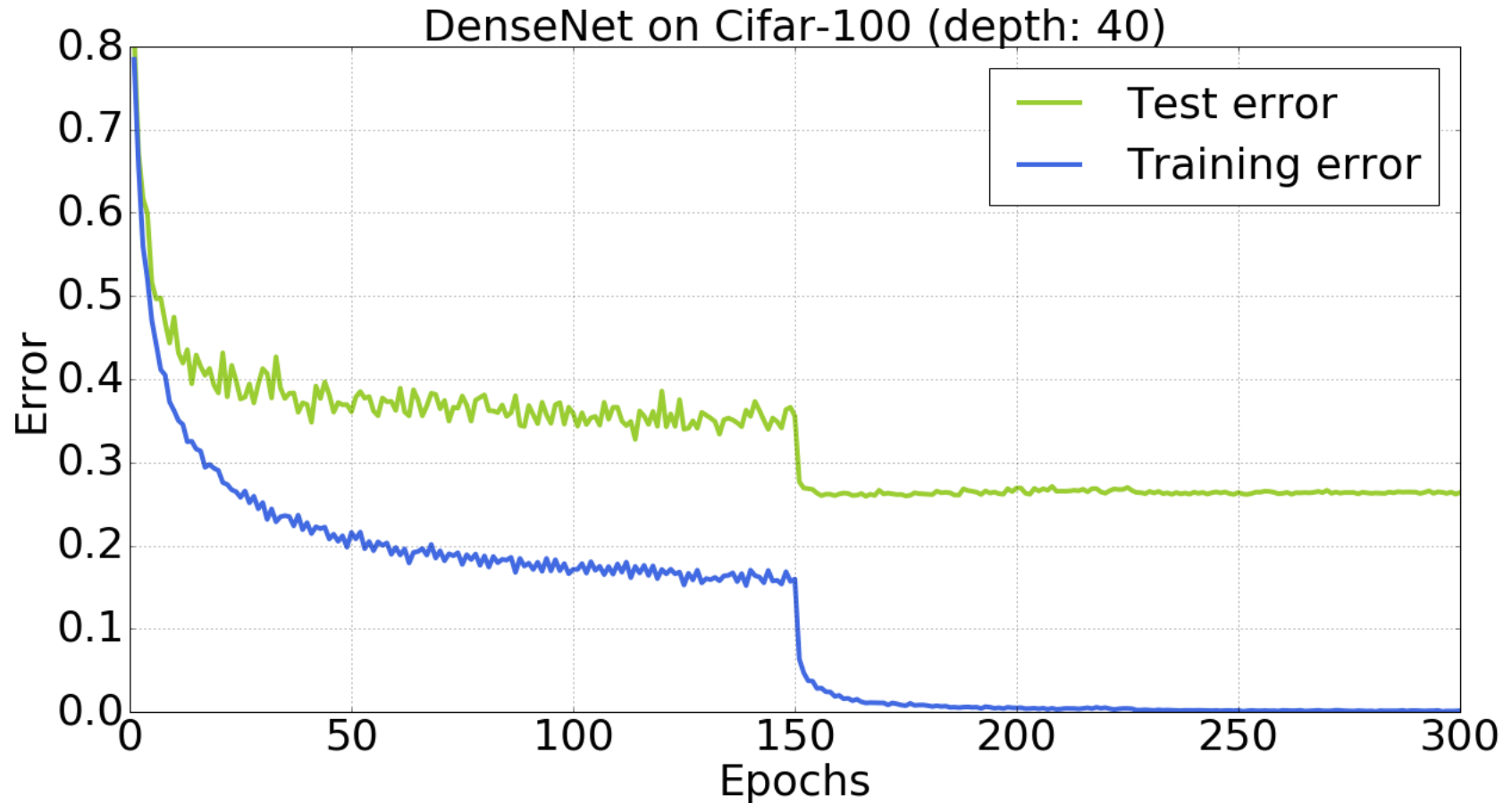
- Stochastic learning/training: a small subset of samples from the training set are propagated through network and then a parameter update is performed

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \tilde{\nabla}_{\mathbf{w}} R_{\text{emp}}(\mathbf{y}, \mathbf{X}, \mathbf{w})|_{\mathbf{w}^{\text{old}}}$$

Approx. the full-batch gradient with a random subsample (“mini batch”):

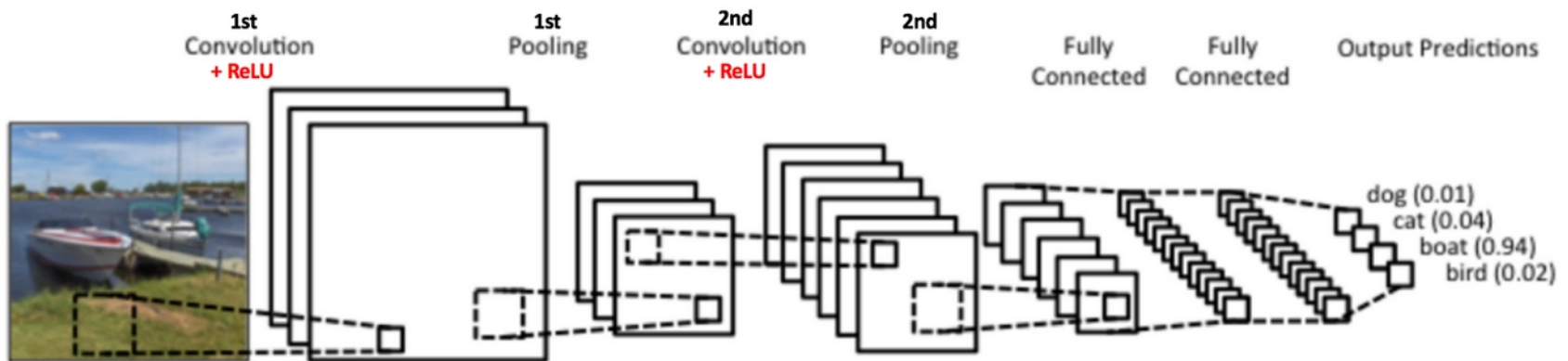
$$\nabla_{\mathbf{w}} R_{\text{emp}}(\mathbf{y}, \mathbf{X}, \mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \nabla_{\mathbf{w}} L(y^n, g(\mathbf{x}^n, \mathbf{w})) \approx \frac{1}{B} \sum_{b=1}^B \nabla_{\mathbf{w}} L(y^{n_b}, g(\mathbf{x}^{n_b}, \mathbf{w}))$$

Epochs, batch size, learning curves

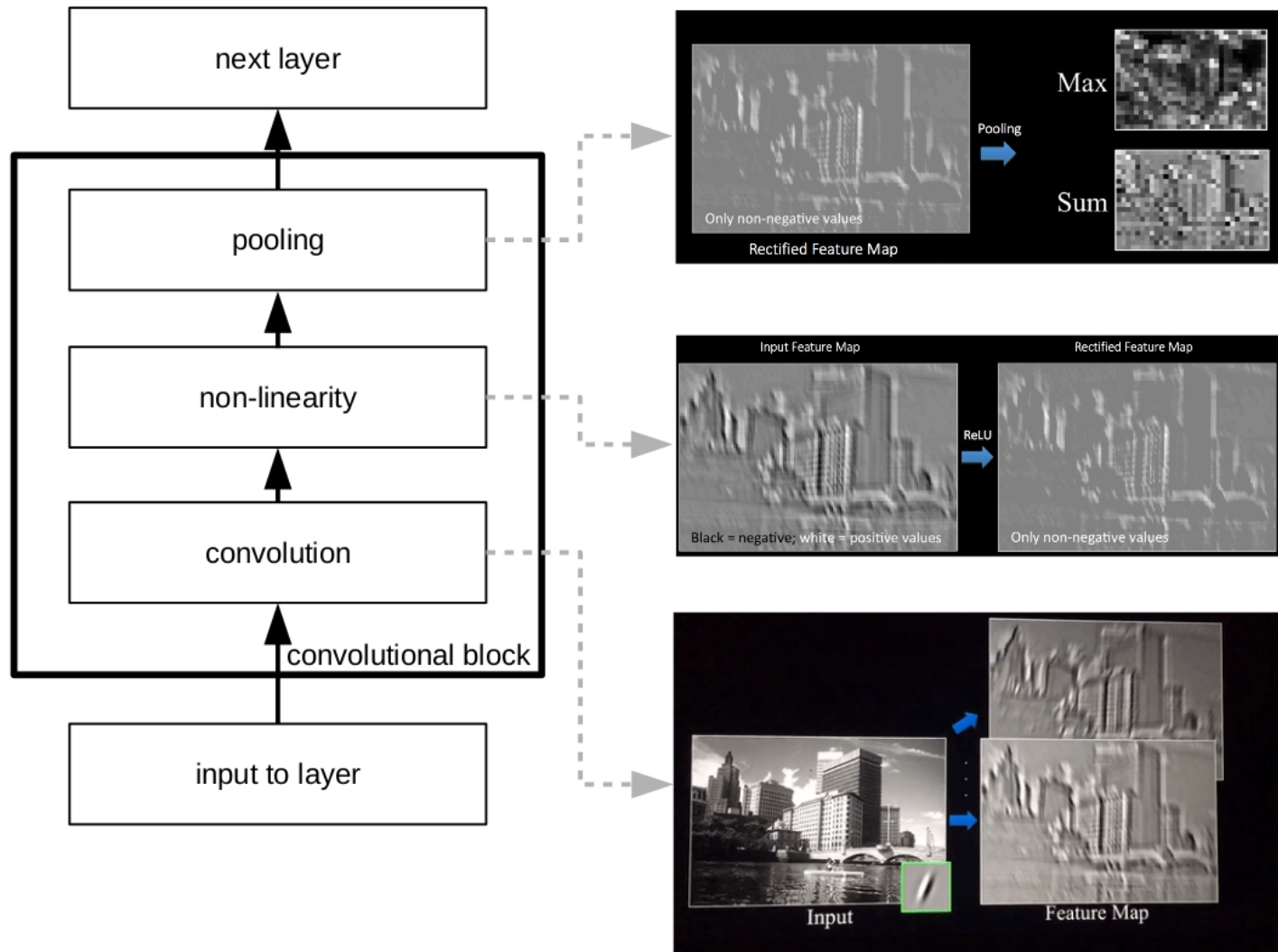


Recap: CNNs

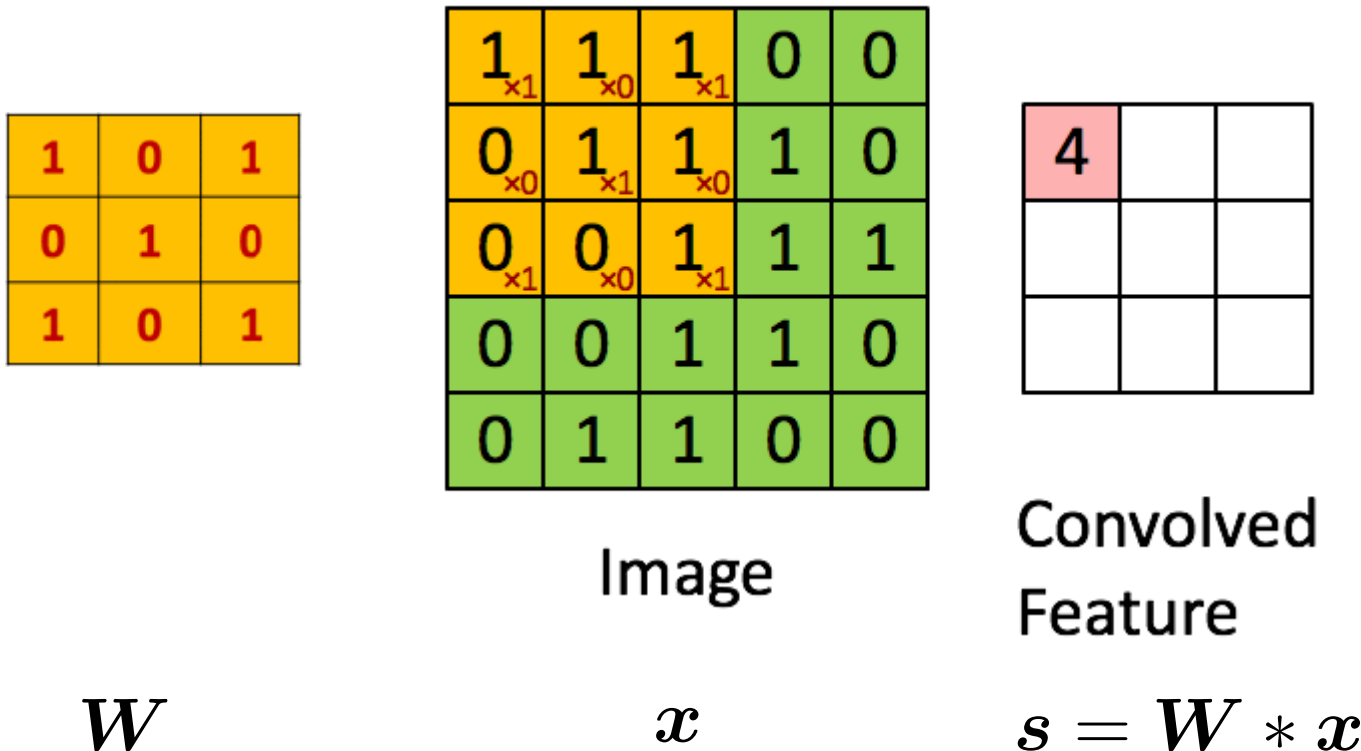
- Typically multiple steps of convolution and pooling
- Recently: strided convolutions instead of pooling



Convolutional blocks: Core elements of a CNN



CNNs: The convolution operation explained



Images and the convolution operation (1/2)

$$(h * k)(a, b) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} h(a + i, b + j) k(i, j).$$

- We identify the image with the function h :
image $x \in \mathbb{R}^{H \times W}$ with height H and width W
- We identify the kernel function k with:
a weight matrix $W \in \mathbb{R}^{R \times R}$ with size R
(quadratic only for simplicity)

Images and the convolution operation (2/2)

- The weight matrix, also called kernel, is “shifted” across the image to obtain a *feature map*:

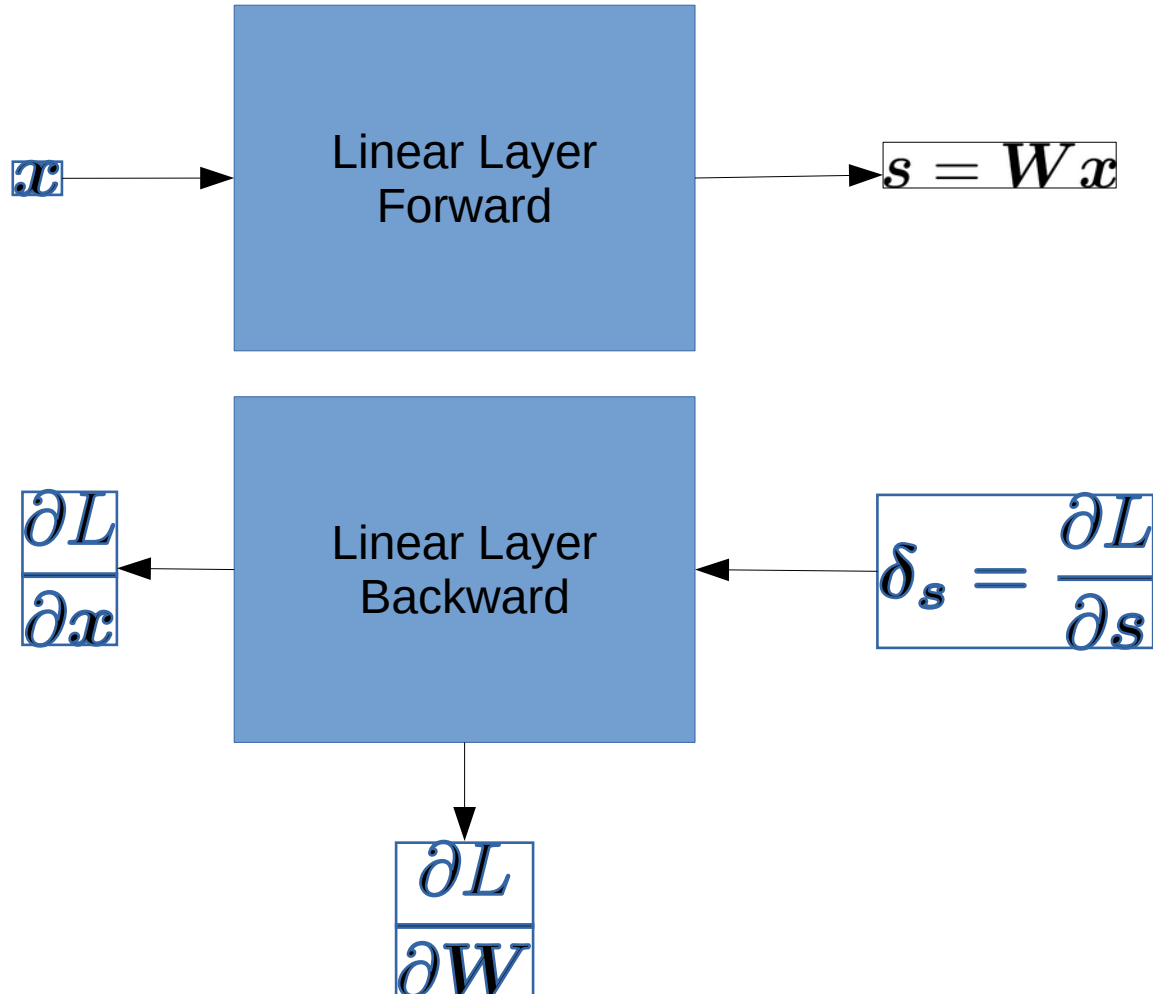
$$s_{a,b} = \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j} x_{a+i,b+j} = (\mathbf{W} * \mathbf{x})_{a,b}.$$

- The quantity $s_{a,b}$ can be, again, seen as pre-activation.

- The convolution operation is essentially a linear operation

Note: indices start with 0 in this chapter!!!

Structure of forward and backward pass in FCN



Derivative w.r.t. inputs

$$s_{a,b} = \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j} x_{a+i,b+j} = (\mathbf{W} * \mathbf{x})_{a,b}.$$

$$\begin{aligned}
 \frac{\partial L}{\partial x_{a,b}} &= \sum_{i=0}^{A-R} \sum_{j=0}^{B-R} \frac{\partial L}{\partial s_{i,j}} \frac{\partial s_{i,j}}{\partial x_{a,b}} \\
 &= \sum_{i=0}^{A-R} \sum_{j=0}^{B-R} \underbrace{\frac{\partial L}{\partial s_{i,j}}}_{:=\delta_{i,j}} \sum_{u=0}^{R-1} \sum_{v=0}^{R-1} w_{u,v} \frac{\partial x_{i+u,j+v}}{\partial x_{a,b}} \\
 &= \sum_{i=0}^{A-R} \sum_{j=0}^{B-R} \delta_{i,j} \sum_{u=0}^{R-1} \sum_{v=0}^{R-1} w_{u,v} \mathbf{1}_{(i+u=a, j+v=b)} \\
 &= \sum_{i=a-R+1}^a \sum_{j=b-R+1}^b \delta_{i,j} w_{a-i, b-j} \\
 &= \sum_{i'=0}^{R-1} \sum_{j'=0}^{R-1} w_{R-1-i', R-1-j'} \delta_{a+i'-R+1, b+j'-R+1}, \\
 &= \sum_{i'=0}^{R-1} \sum_{j'=0}^{R-1} \hat{w}_{i', j'} \hat{\delta}_{a+i', b+j'} \\
 &= (\hat{\mathbf{W}} * \hat{\boldsymbol{\delta}})_{a,b}
 \end{aligned}$$

Derivative w.r.t. weights

$$s_{a,b} = \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j} x_{a+i,b+j} = (\mathbf{W} * \mathbf{x})_{a,b}.$$

$$\begin{aligned} \frac{\partial L}{\partial w_{i,j}} &= \sum_{a=0}^{A-R} \sum_{b=0}^{B-R} \frac{\partial L}{\partial s_{a,b}} \frac{\partial s_{a,b}}{\partial w_{i,j}} \\ &= \sum_{a=0}^{A-R} \sum_{b=0}^{B-R} \underbrace{\frac{\partial L}{\partial s_{a,b}}}_{:=\delta_{a,b}} x_{a+i,b+j} \\ &= (\boldsymbol{\delta} * \mathbf{x})_{i,j} \end{aligned}$$

Backprop explained with 1-D convolutions

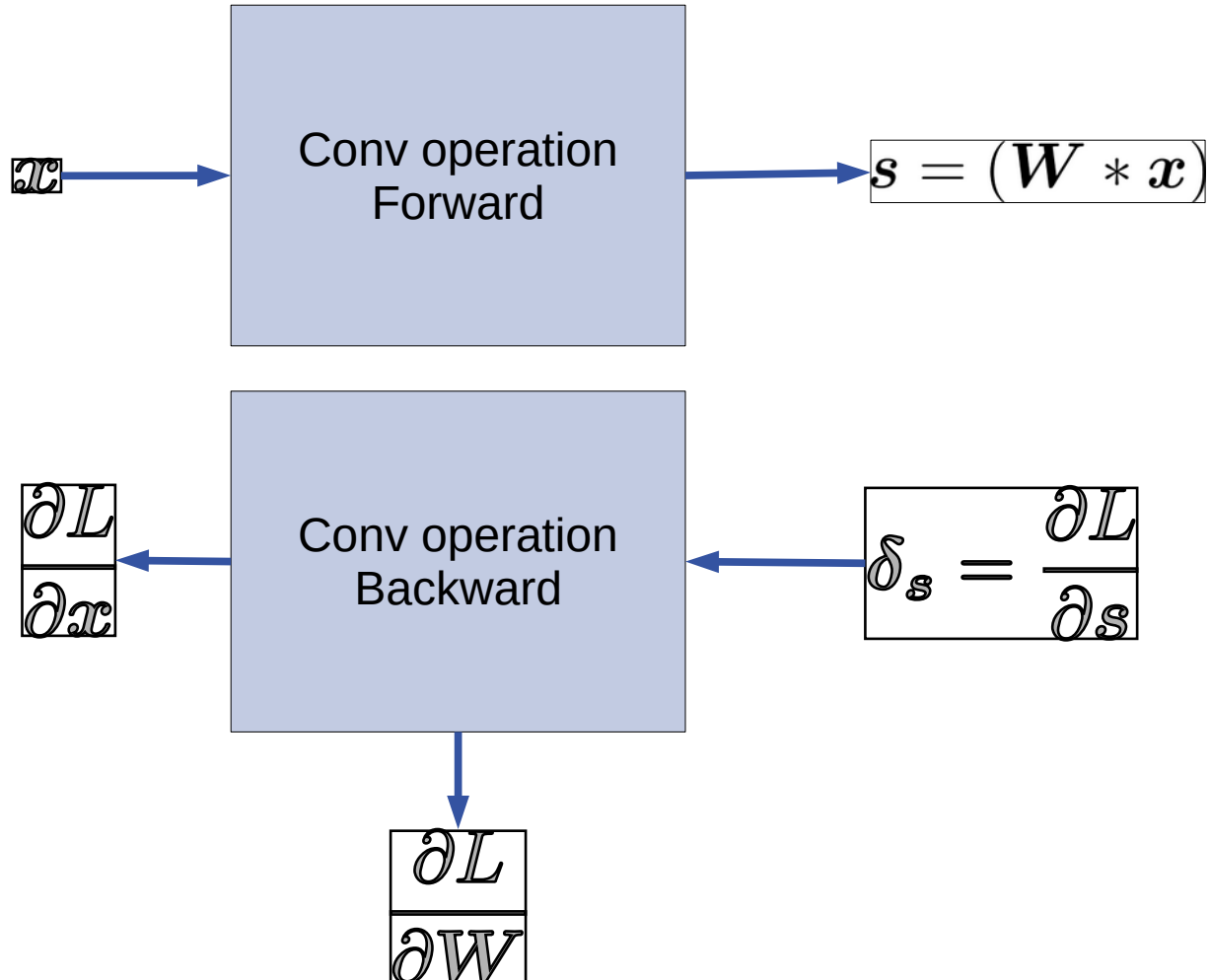
In the backward pass, these become deltas

	s_{-2}	s_{-1}	s_0	s_1	s_2	s_3	s_4	s_5	s_6
x_0	w_2	w_1	w_0						
x_1		w_2	w_1	w_0					
x_2			w_2	w_1	w_0				
x_3				w_2	w_1	w_0			
x_4					w_2	w_1	w_0		
x_5						w_2	w_1	w_0	
x_6							w_2	w_1	w_0

Forward pass

Backward pass

Structure of forward and backward pass in CNNs



Remark 1: Size of feature maps

- The dimensions of the feature maps are:

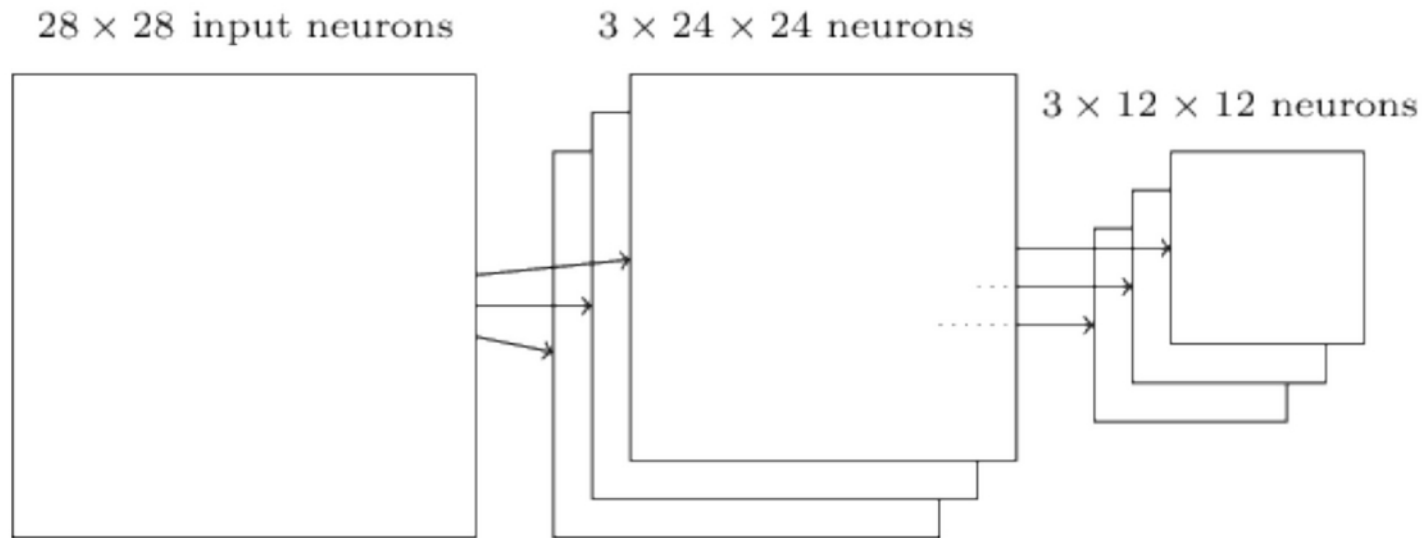
$$A^{[l+1]} \times B^{[l+1]} \times D^{[l+1]},$$

with $A^{[l+1]} = (A^{[l]} - R + 2P)/S + 1$ and $B^{[l+1]} = (B^{[l]} - R + 2P)/S + 1$, where

- $A^{[l]}$ is the height of the feature maps or the input image in the previous layer
- $B^{[l]}$ is the width of the feature maps or the input image in the previous layer
- P is the amount of (symmetric) zero padding,
- R is the kernel size,
- S is the stride, and
- $D^{[l+1]}$ is the number of output feature maps of the convolutional layer.

Remark 2: Multiple filters

- We want to find different features
 - Thus, we need multiple filters
 - Each filter generates a feature map

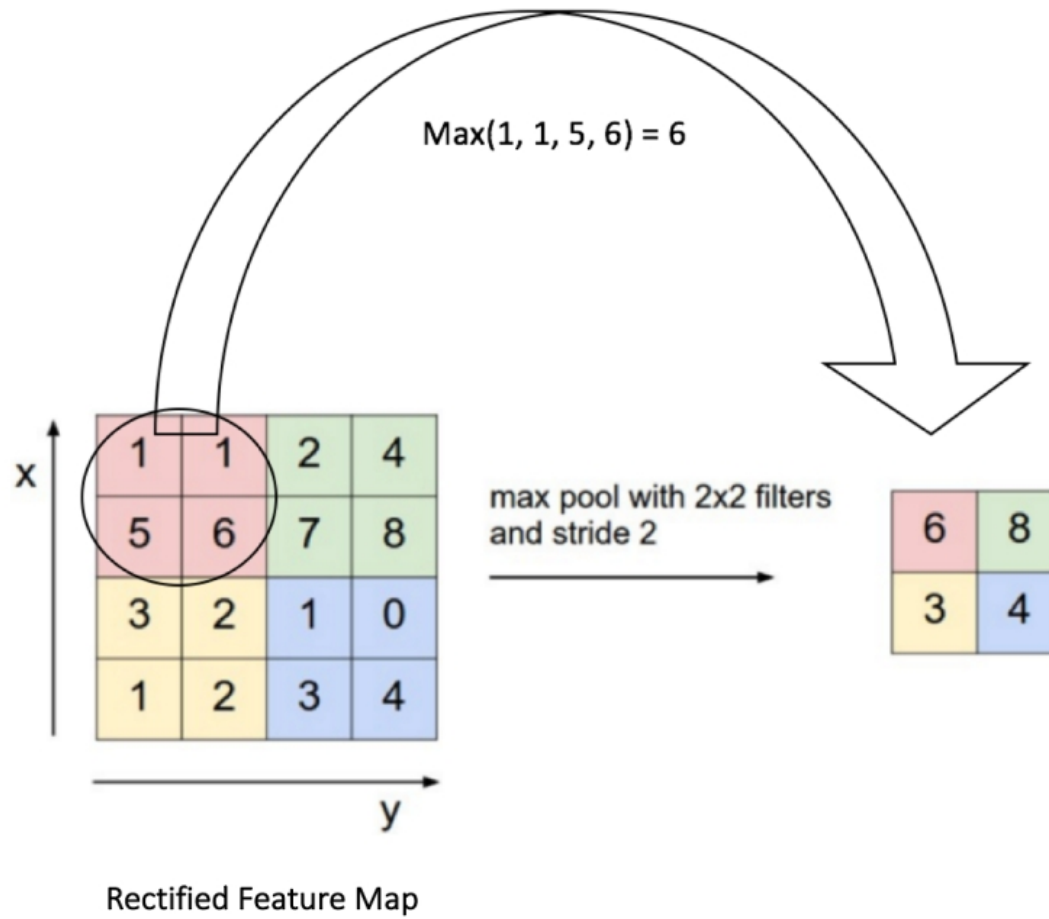


Remark 3: Images with depth

- Typically, we have 3 (RGB) input channels
 - Grayscale: 1 channel
 - Channels: additional dimension (W, H, C)
- In hidden conv layers, feature maps correspond to channels
- How do we deal with it with CNNs?

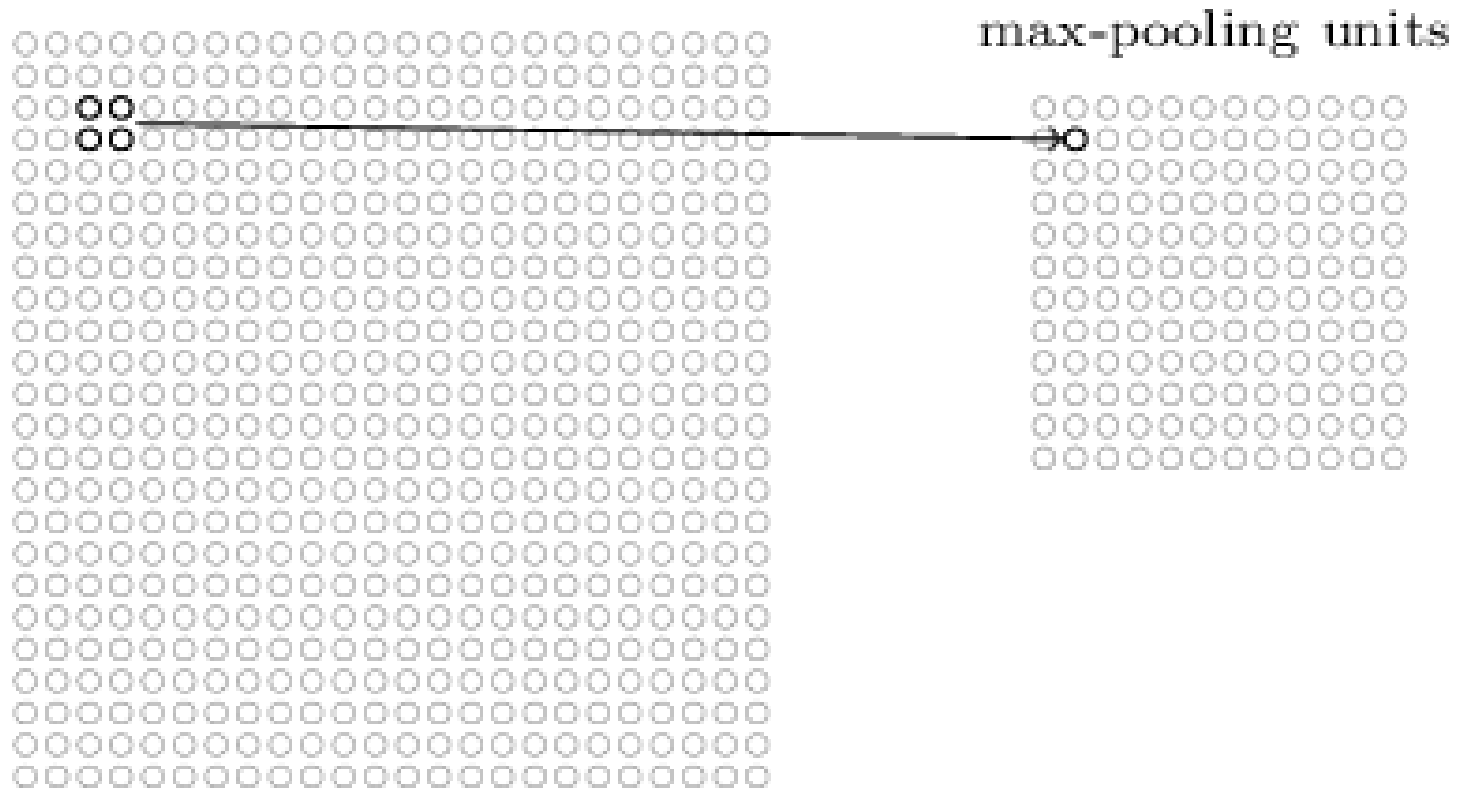
$$s_{a,b} = \sum_{c=0}^{C-1} \sum_{i=0}^{R-1} \sum_{j=0}^{R-1} w_{i,j,c} x_{a+i,b+j,c} = (\mathbf{W} * \mathbf{x})_{a,b}$$

CNNs: The max-pooling operation

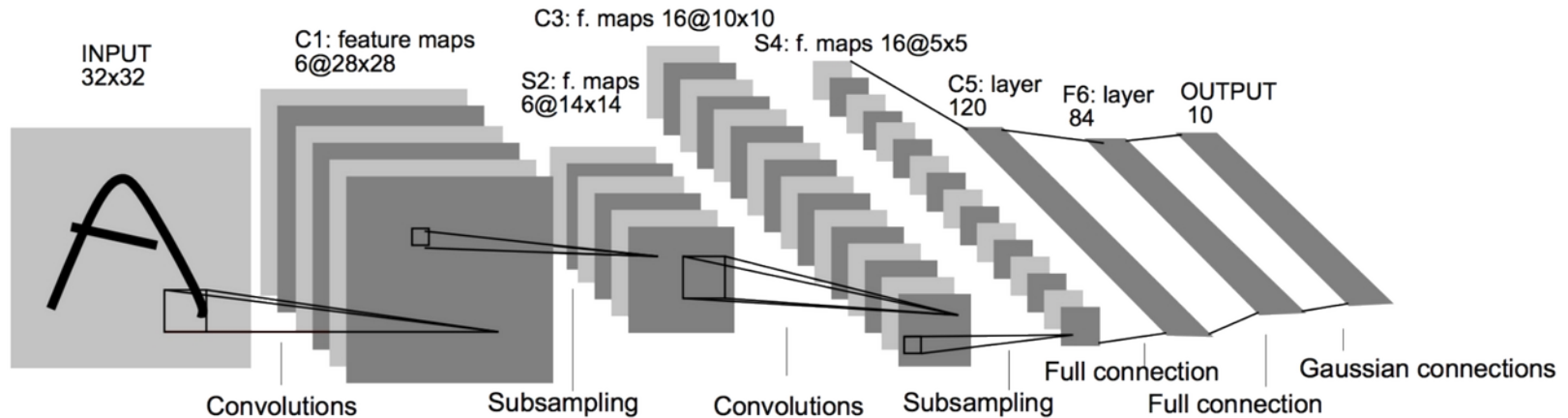


CNNs: The max-pooling operation

hidden neurons (output from feature map)

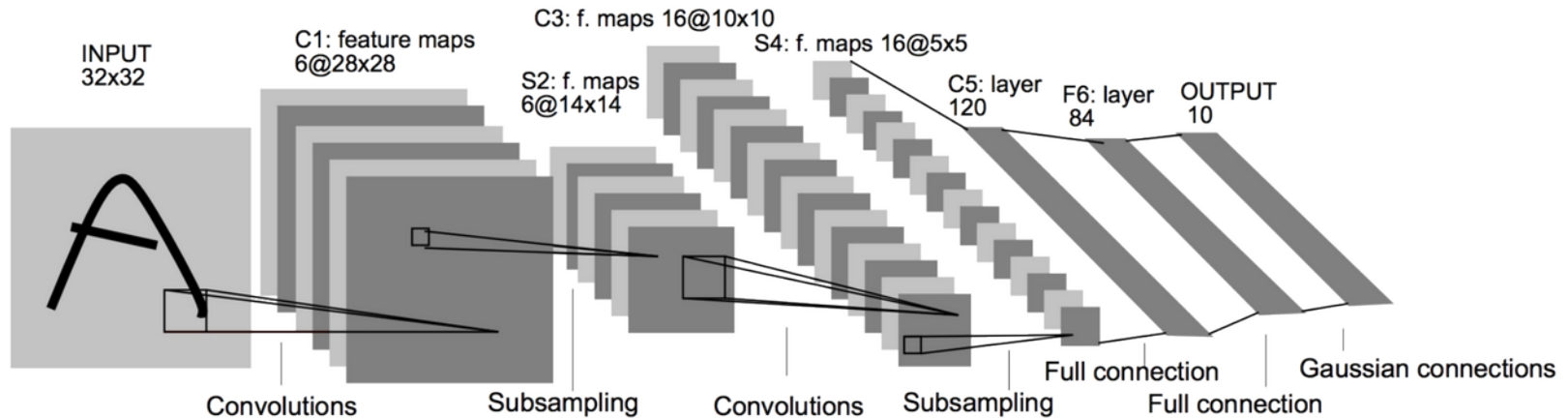


Example: LeNet



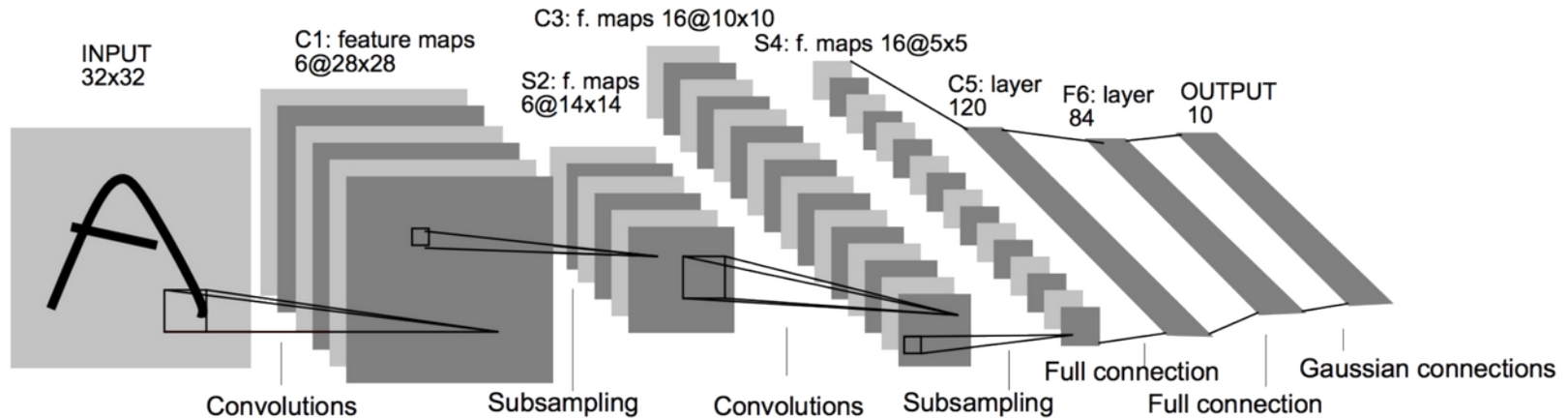
- **Input:** gray scale image of size 32x32 or 32x32x1
- **C1:** ConvLayer with 6 kernels of size (5,5) and stride 1
 - Output dimensions are: 28x28x6
 - Number of trainable parameters: $(1*5*5+1)*6=156$
- **S2:** Pooling with size (2,2) and stride 2.
 - Output dimensions are: 14x14x6

Example: LeNet



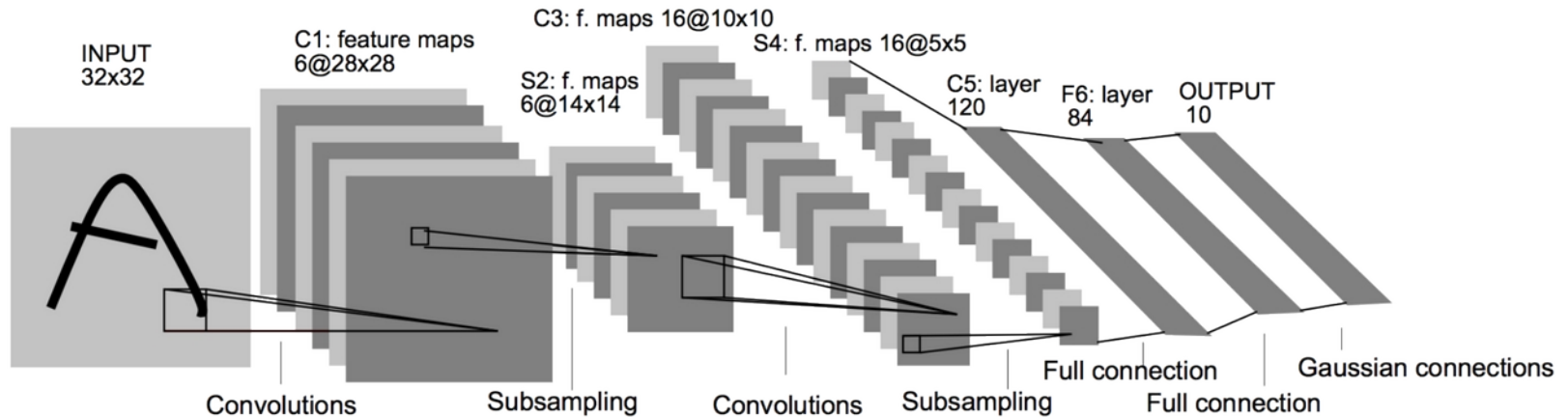
- **C3:** ConvLayer with 16 kernels of size (5,5) and stride 1
 - Output dimensions are: 10x10x16
 - Number of trainable parameters: $(6*5*5+1)*16=2416$
- **S4:** Pooling with size (2,2) and stride 2.
 - Output dimensions are: 5x5x16

Example: LeNet



- **C5:** ConvLayer with 120 kernels of size (5,5) and stride 1
 - Output dimensions are: 1x1x120
 - Practically means “full-connection”
 - Number of trainable parameters: $(16*5*5+1)*120=48120$
- **F6:** Fully-connected layer with 84 units.
 - Number of trainable parameters: $(120+1)*84$

Example: LeNet



- **Output:** Fully-connected layer with 10 units.
 - Number of trainable parameters: $(84+1)*10$

Regularization

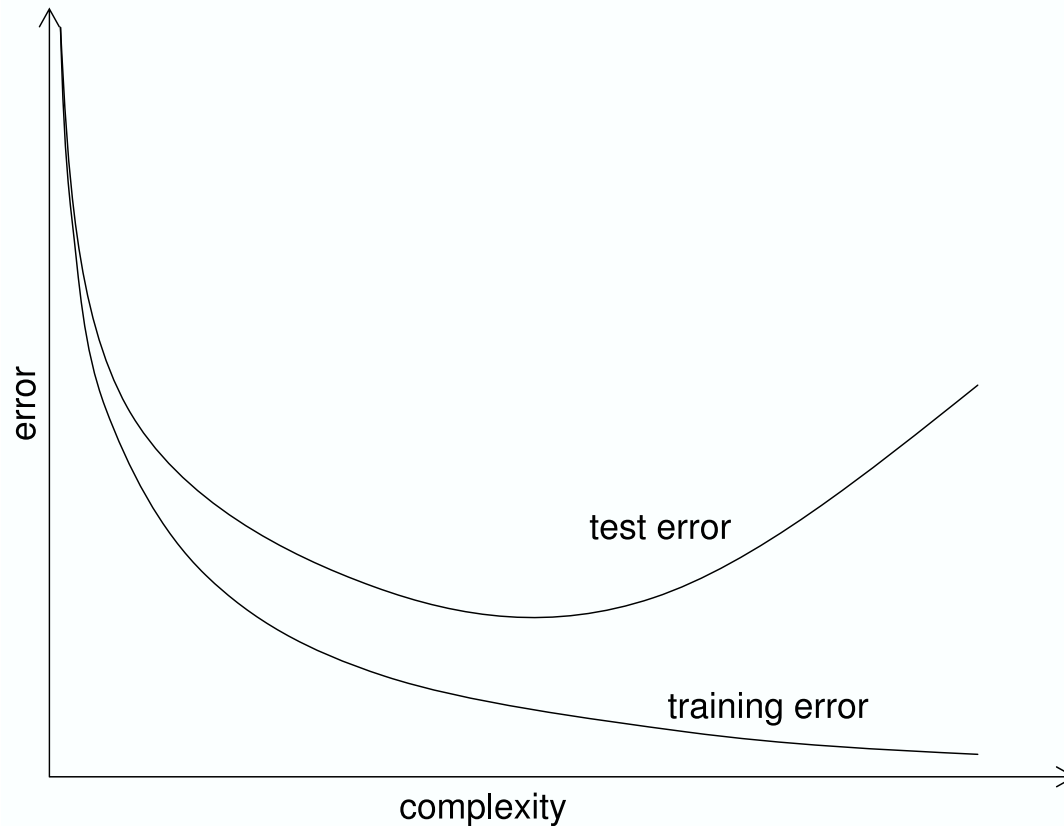
- 8.1 Weight Decay
- 8.2 Neuron Noise and Weight Noise
- 8.3 Weight Sharing
- 8.4 Dropout
- 8.5 Multi-task Learning
- 8.6 Growing
- 8.7 Pruning Methods

Regularization

- 8.8 Early Stopping
- 8.9 Flat Minimum Search
- 8.10 Data Augmentation
- 8.11 Self-regularization by Stochastic Gradient Descent

Risk as a Function of the Complexity

$$R \leq R_{\text{emp}} + \text{complexity}$$



Principle Regularization Approaches

Typical **regularization terms** are:

- Smoothing terms, which control the curvature and higher order derivatives of the function represented by the network
- Complexity terms, which control number of units and weights or their precision

Types:

- Regularization during training
- Regularization after training

8.1 Weight Decay

The empirical risk R_{emp} is extended by a complexity term $\Omega(w)$:

$$R_{reg}(w) = R_{emp}(w) + \lambda \Omega(w)$$

Gradient descent:

$$\nabla_w R_{reg}(w) = \nabla_w R_{emp}(w) + \lambda \nabla_w \Omega(w)$$

L2 weight decay:

$$R_{reg}(w) = R_{emp}(w) + \frac{\lambda}{2} \|w\|_2^2$$

8.1 Weight Decay

Interpretation of L2 weight decay

If we neglect the first data-driven term and only consider the complexity term, the gradient would be λw .

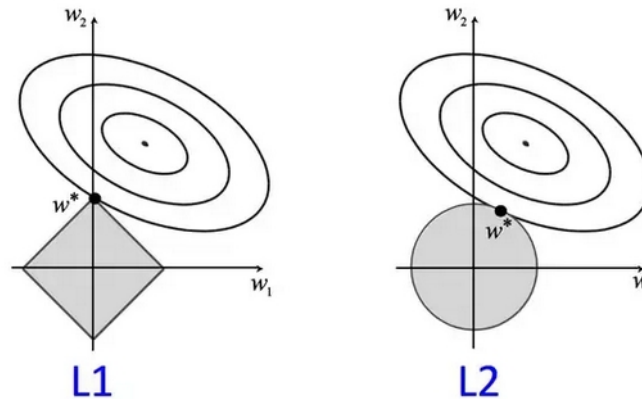
$$w^{(t+1)} = w^{(t)} - \eta \lambda w^{(t)}$$

$$w^{(t)} = (1 - \eta \lambda)^t w^{(0)}$$

The weights decay to zero, if $0 < \eta \lambda < 1$.

8.1 Weight Decay

L1 weight decay: $R_{\text{reg}}(\mathbf{w}) = R_{\text{emp}}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1$



Further variants:

- $\Omega(\mathbf{w}) = \log(1 + \mathbf{w}^T \mathbf{w})$
- $\Omega(\mathbf{w}) = \sum_i \frac{w_i^2 / w_0^2}{1 + w_i^2 / w_0^2}$, w_0 : threshold

8.1 Weight Decay: Interpretation (second part)

Assuming a quadratic approximation of the error around a local minimum w_0 :

$$R(w) = R(w_0) + \frac{1}{2}(w - w_0)^T H(w - w_0)$$

Minimum: $w = w_0$

Adding a L2 regularization term, the minimum moves to a point $\tilde{w}$, where: $\lambda \tilde{w} + H(\tilde{w} - w_0) = 0$. We then solve for $\tilde{w}$. We can now use the *eigendecomposition* $H = Q\Lambda Q^T$:

$$\tilde{w} = (H + \lambda I)^{-1} H w_0$$

$$\tilde{w} = Q(\Lambda + \lambda I)^{-1} \Lambda Q^T w_0$$

Thus eigenvalues are scaled according to $\frac{\lambda_i}{\lambda_i + \lambda}$.

8.2 Neuron Noise and Weight Noise

To regularize and to avoid that the network extracts individual characteristics of few training examples noise can be added to:

- the training examples,
- the network components.

Properties:

- Closely related to L2 regularization (if noise amplitude is small)
- involves second derivatives of the network function, and so evaluation of the gradients of this error with respect to network weights will be computationally demanding.

Bishop, C. M. (1993). Curvature-driven smoothing: a learning algorithm for feedforward networks. IEEE Transactions on Neural Networks, 4(5), 882-884.
Bishop, C. M. (1995). Neural networks for pattern recognition. Oxford university press. [Section 9.3]

8.3 Weight Sharing

Nowlan and Hinton propose that the weights have special preferred values and should be grouped: *soft weight sharing*

Each group can be modeled by a Gaussian:

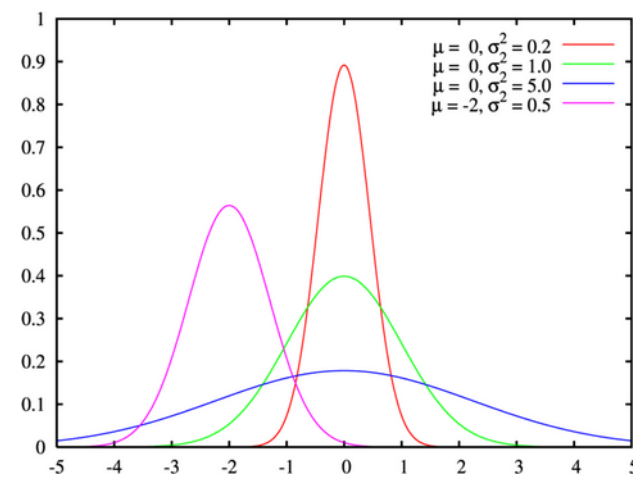
$$G(w, \sigma_j, \mu_j) = \frac{1}{(2\pi)^{1/2} \sigma_j} \exp \left(-\frac{1}{2\sigma_j^2} (w - \mu_j)^2 \right).$$

Each group j has a prior size $p(j) = \alpha_j$ which estimates what percentage of weights will belong to this group.

Likelihood of a weight w :

$$p(w) = \sum_j \alpha_j G(w, \sigma_j, \mu_j)$$

$$p(w | j) = \frac{\alpha_j G(w, \sigma_j, \mu_j)}{\sum_k \alpha_k G(w, \sigma_k, \mu_k)}$$



8.3 Weight Sharing

$$\begin{aligned} R(\mathbf{w}) &= R_{\text{emp}}(\mathbf{w}) - \lambda \sum_i \log p(w_i) \\ &= R_{\text{emp}}(\mathbf{w}) - \lambda \sum_i \log \left(\sum_j \alpha_j G(w_i, \sigma_j, \mu_j) \right) \end{aligned}$$

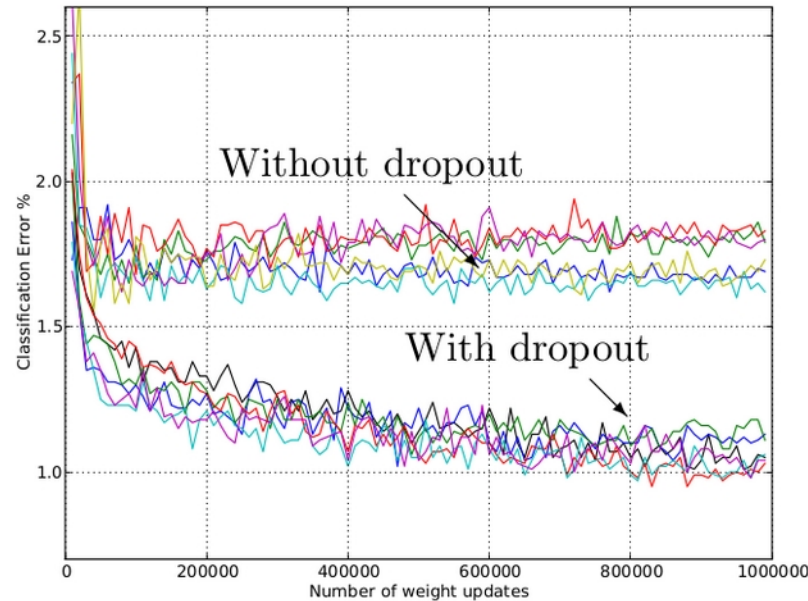
$$\frac{\partial R(\mathbf{w})}{\partial w_i} = \frac{\partial R_{\text{emp}}(\mathbf{w})}{\partial w_i} + \lambda \sum_j p(w_i | j) \frac{(w_i - \mu_j)}{\sigma_j^2}$$

8.3 Weight Sharing

- Weight sharing is also a main component of CNNs
 - Neurons in the same feature map share their weights
 - One reason for the success of CNNs

8.4 Dropout

- Main idea: Set neuron activations to zero randomly during training
- Standard neural network: $a = f(Wx)$



Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. The journal of machine learning research, 15(1), 1929-1958.

8.4 Dropout

- Main idea: Set neuron activations to zero randomly during training

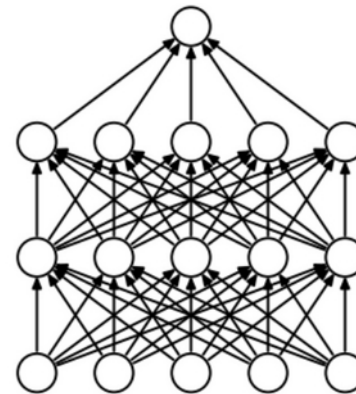
- Neural network with dropout: $a = d \odot f(Wx)$

dropout mask d :

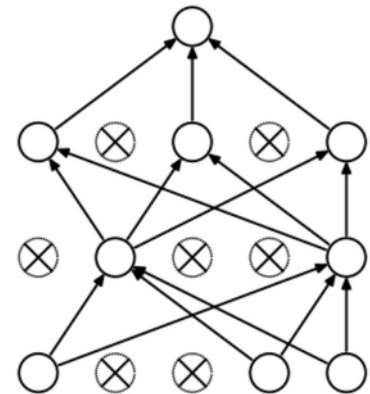
- Realization of drawing from an I-dimensional Bernoulli RV

- Probability: $p : d_i \sim \mathcal{B}(1, p)$

$$\mathbb{E}(d_i) = p, \text{Var}(d_i) = p(1 - p)$$



(a) Standard Neural Net



(b) After applying dropout.

- “keep-prob”: probability to keep a unit is p

8.4 Dropout: Pseudo-code forward pass

```
def forward_pass(x, w_list, b_list):  
    w1, w2, w3 = w_list  
    b1, b2, b3 = b_list  
  
    z1 = np.dot(x, w1.T) + b1  
    h1 = np.maximum(0, z1)  # ReLU  
    h1 *= np.random.binomial(1, 0.5, size=h1.shape)  # 50% dropout  
  
    z2 = np.dot(h1, w2.T) + b2  
    h2 = np.maximum(0, z2)  
    h2 *= np.random.binomial(1, 0.5, size=h2.shape)  # 50% dropout  
  
    z3 = np.dot(h2, w3.T) + b3  
    h3 = softmax(z3)  
    return (h1, h2, h3)
```

8.4 Dropout: introduced bias

- Dropout should only be applied during training, but not for inference
- Network with large dropped parts (e.g. “training time”):

$$\tilde{s} = \mathbf{w}^T (\mathbf{d} \odot \mathbf{a})$$

- Network without dropped parts (e.g. “test time”):

$$\mathbf{s} = \mathbf{w}^T \mathbf{a}$$

$$\mathbb{E}_{p_{\text{data}}, d_i}(\tilde{s}) = \mathbb{E}_{p_{\text{data}}, d_i} \left(\sum_i w_i d_i a_i \right) = p \sum_i w_i \mathbb{E}_{p_{\text{data}}}(a_i)$$

$$\mathbb{E}_{p_{\text{data}}}(\mathbf{s}) = \sum_i w_i \mathbb{E}_{p_{\text{data}}}(a_i)$$

8.4 Dropout: bias countering strategies

Strategies to counter this bias:

- Weight scaling:
 - After training, weights are rescaled by p to remove the bias
- State scaling:
 - During training, activations are re-scaled by $1/p$
 - Advantage: network can be left as it is; the dropout layer has to have two modes: training and test mode. In training mode, outgoing values have to be rescaled.
- Monte-Carlo-Dropout:
 - Multiple forward-passes with dropout are averaged

8.4 Dropout: DO as Bayesian approximation

Not for
EXAM

- Objective with dropout regularization:

$$R_{\text{Dropout-reg}}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N L(y_n, \hat{y}_n) + \lambda \sum_{l=1}^L \|\mathbf{w}^{[l]}\|^2$$

where $\hat{y}_n$ are the predictions of a network that applies dropout.

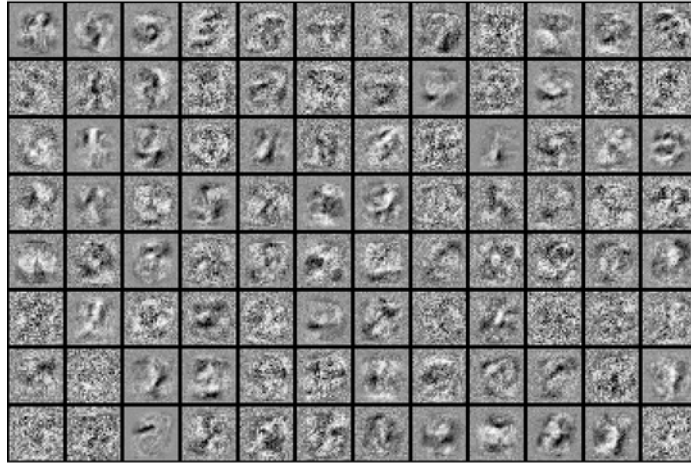
- In a Bayesian setting with distributions over parameters, and Gaussian process approximation, we can obtain:

$$R_{\text{GP-MC}} \propto \frac{1}{N} \sum_{n=1}^N -\frac{1}{\tau} \log p(y_n | \mathbf{x}_n, \hat{\mathbf{w}}_n) + \sum_{l=1}^L \frac{p_i l^2}{2\tau N} \|\mathbf{M}_i\|^2$$

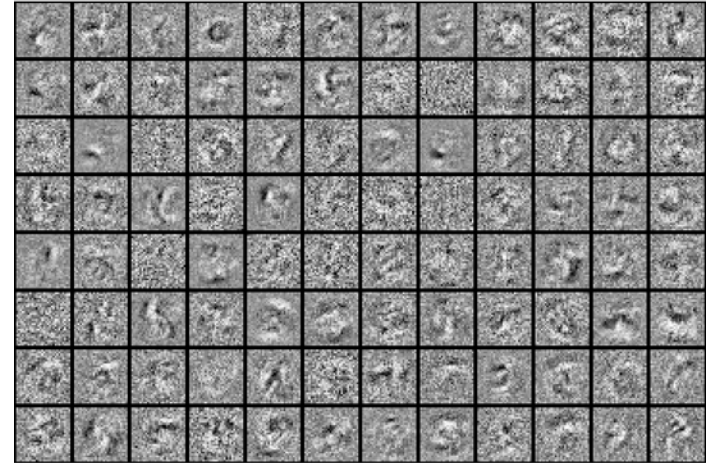
From which the training objective above can be recovered with suitable choice of τ, l .

Gal, Y., & Ghahramani, Z. (2016, June). Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In international conference on machine learning (pp. 1050-1059).

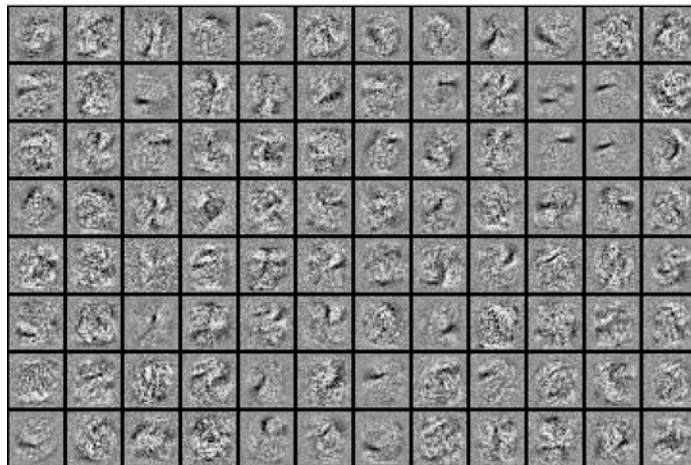
8.4 Dropout: Effects of dropout



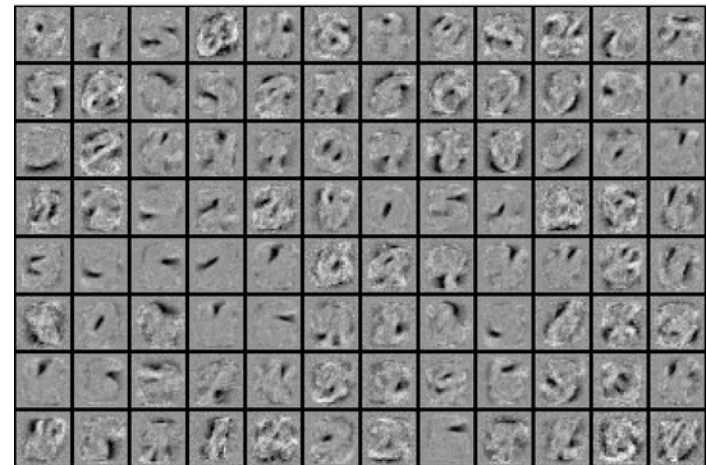
a) Sigmoid activations



a) ReLU activations

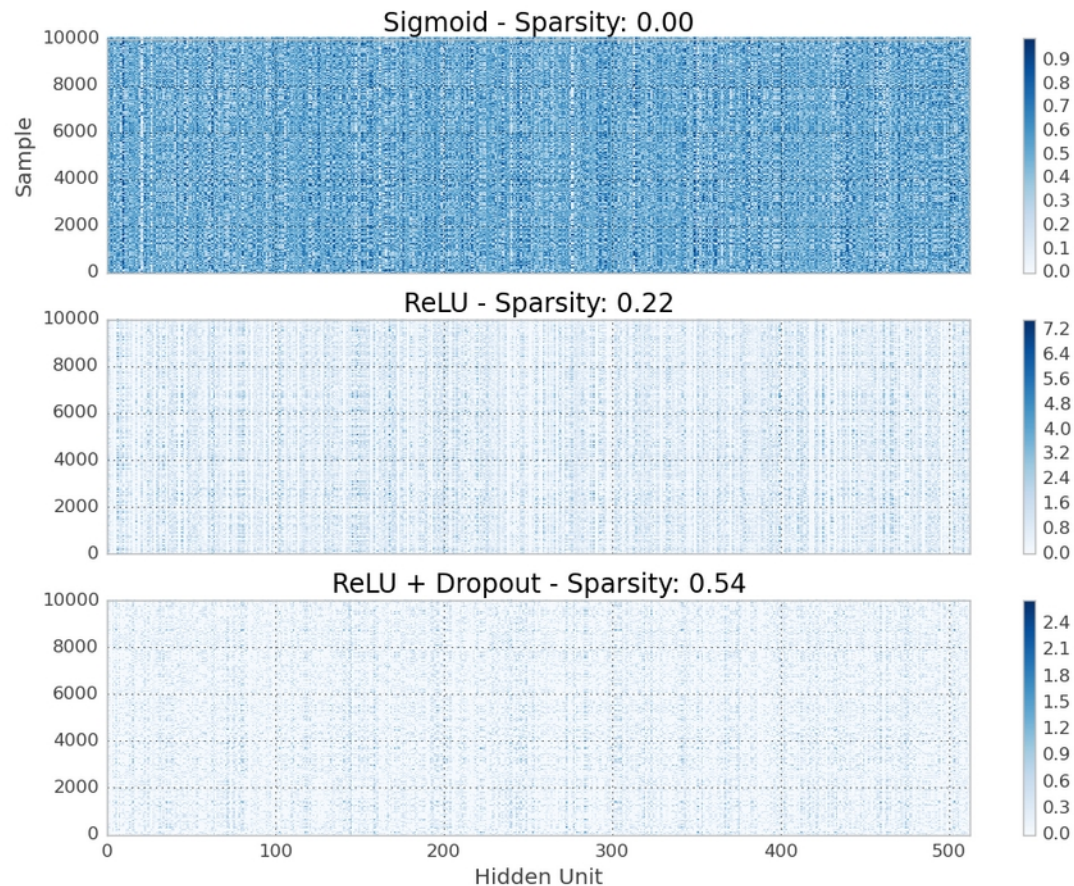


c) ReLU activations, DO



d) ReLU activations, DO,
input DO

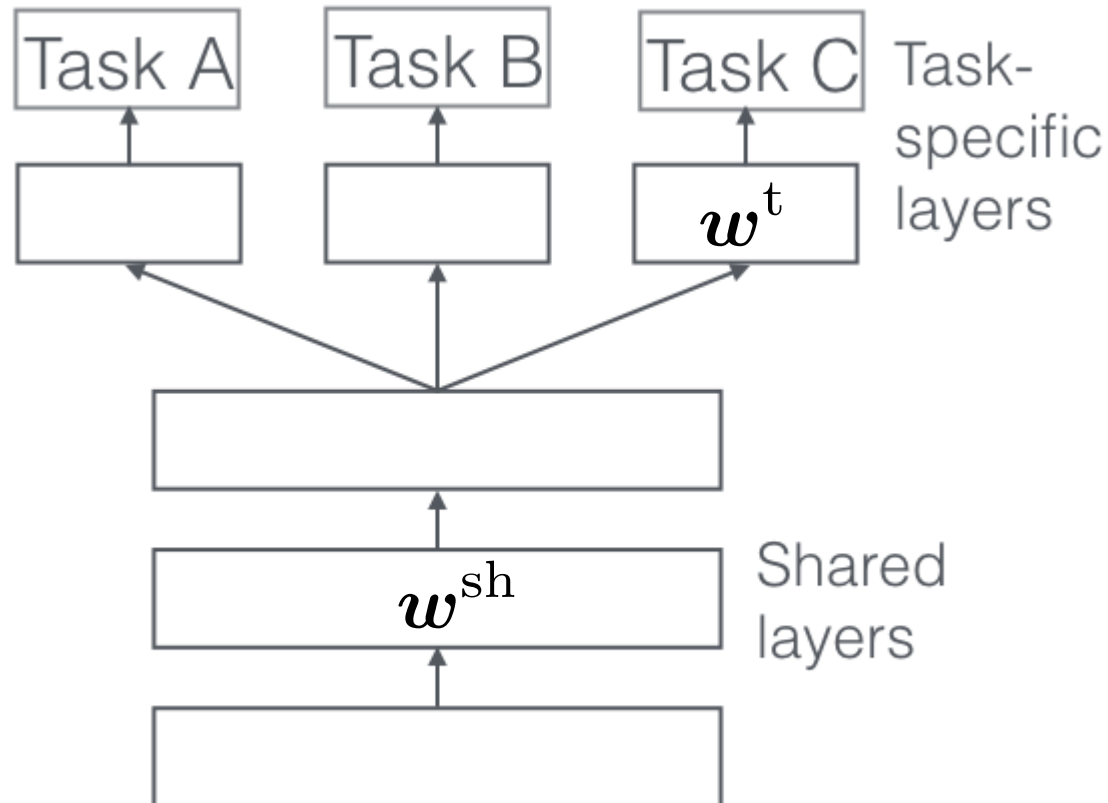
8.4 Dropout: Effects of dropout on sparsity



8.5 Multi-task Learning

- Main idea: Solve multiple prediction tasks with a single network with multiple output units
- Multi-task Learning is an approach to inductive transfer that improves generalization by using the domain information contained in the training signals of related tasks as an inductive bias. It does this by learning tasks in parallel while using a shared representation; what is learned for each task can help other tasks be learned better. (Caruana, 1997)
- Multi-task setting: $g(x, w^{\text{sh}}, w^t) : \mathcal{X} \mapsto \mathcal{Y}^t$
- For each task there can be a task specific loss function $L^t(\cdot, \cdot)$.

8.5 Multi-task learning



8.5 Multi-task Learning

- With the usual empirical risk minimization principle the multi-task objective has following form:

$$\min_{\mathbf{w}^{\text{sh}}, \mathbf{w}^1, \dots, \mathbf{w}^t, \dots, \mathbf{w}^T} \sum_{t=1}^T \alpha^t R_{\text{emp}}^t(\mathbf{w}^{\text{sh}}, \mathbf{w}^t)$$

- Assuming we are mainly interested in task $t = 1$, one can use all other tasks as auxiliary tasks.
- The auxiliary tasks act as regularization for task $t = 1$ via their influence on $\mathbf{w}^{\text{sh}}$.
- Weighting of tasks can be learned or hyperparameters

8.6 Growing

Main idea:

- Start with a small network
- Add new units in a step-wise way or new weights which amount to increasing the complexity

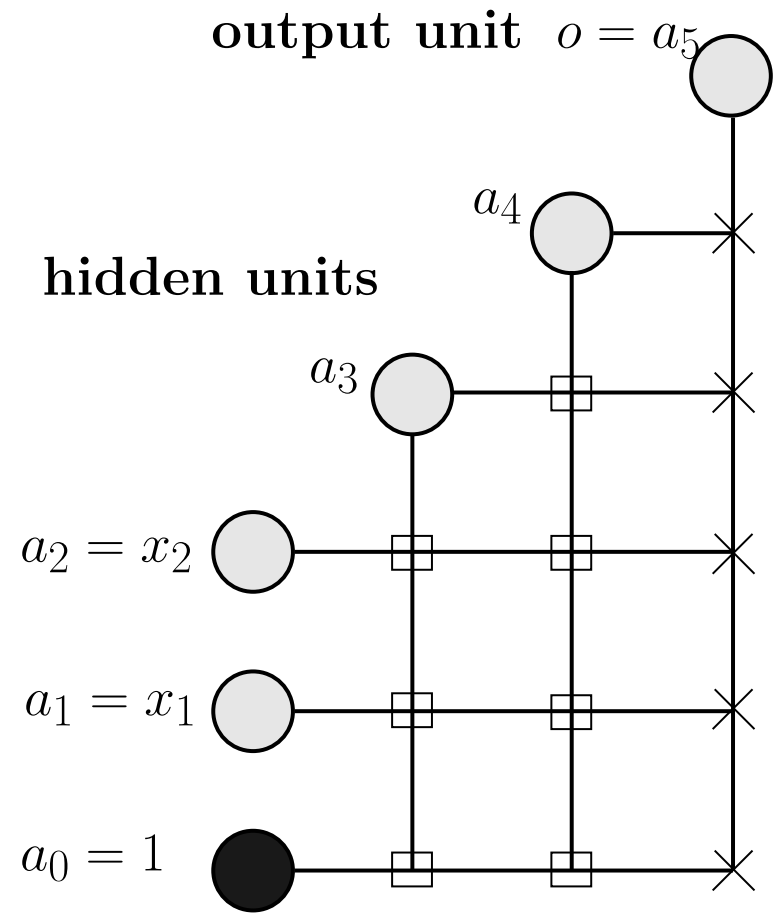
8.6 Growing – Cascade Correlation

- Start: Single layer network
- Add sequentially new hidden neurons
- These new hidden neurons have incoming connections from all existing input neurons and hidden neurons.
- These connections are adjusted such that the **correlation between the new unit's activation and the output errors is maximal**.
- Units that correlate with the error signal can potentially help the whole network to decrease the overall error.

8.6 Growing – Cascade Correlation

Architecture of the network:

- Squares are weights which are trained.
- Crosses represent weights which are retrained only after the addition of a hidden unit.



8.6 Growing – Cascade Correlation

Objective to maximize (correlation with error):

$$C_{\xi} = \sum_{k=1}^K \left| \sum_{n=1}^N (a_{\xi}^n - \bar{a}_{\xi}) (\epsilon_k^n - \bar{\epsilon}_k) \right|$$
$$\epsilon_k^n = (g(\mathbf{x}^n; \mathbf{w}) - y_k^n)$$

The new hidden unit ξ gets input from all input neurons and hidden neurons that currently exits:

$$s_{\xi} = \sum_{j=1}^J w_{\xi j} a_j$$

8.6 Growing – Cascade Correlation

Derivative of C_ξ with respect to the weights $w_{\xi i}$:

$$\frac{\partial C_\xi}{\partial w_{\xi j}} = \pm \sum_{k=1}^K \sum_{n=1}^N (\epsilon_k^n - \bar{\epsilon}_k) f'(s_\xi) a_j^n$$

The training of cascade-correlation networks is very fast because

- only the incoming connections to the new hidden units have to be trained.
- the hidden to output weights are then found by training methods for single layer networks

8.7 Pruning Methods

Main idea:

- First train neural network until a local minimum of the training error is found
- Then the complexity of the trained network is reduced by removing weights (setting them to zero) from the network.
- Simple approach: **Pruning away low weights**
 - Remove $p\%$ of lowest weights in each layer.
 - Often results in improved accuracy
 - Smaller networks, faster calculation; e.g. for NN on mobile devices

8.7 Pruning Methods:

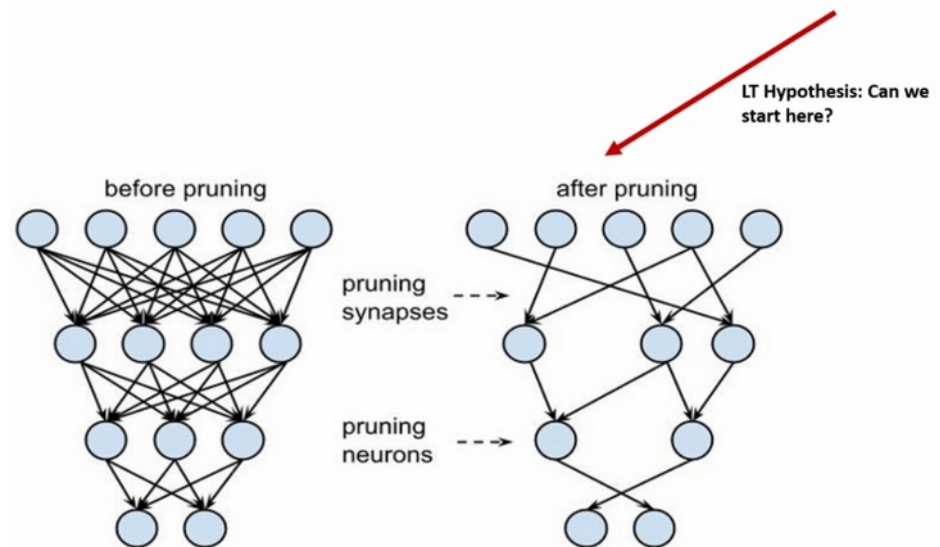
Lottery ticket hypothesis

- **Observation:**

- 90% of a trained network pruned away with simple technique → accuracy remains high
- The 10% remaining connections re-set to initial weights → accuracy still high

- **“lottery ticket hypothesis”:**

- dense, randomly-initialized, feed-forward networks contain subnetworks (“winning tickets”) that - when trained in isolation - reach test accuracy comparable to the original network in a similar number of iterations.
- The winning tickets “have won the initialization lottery”: their connections have initial weights that make training particularly effective.



Frankle, J., & Carbin, M. (2018). The lottery ticket hypothesis: Finding sparse, trainable neural networks. arXiv preprint arXiv:1803.03635.

8.7 Pruning Low-Influence Weights

- A Taylor expansion $R(\mathbf{w})$ of the empirical error around the local minimum $\mathbf{w}^*$ is made.
- The gradient vanishes in the minimum $\nabla_{\mathbf{w}} R(\mathbf{w}^*) = \mathbf{0}$, therefore we obtain with $\Delta \mathbf{w} = (\mathbf{w} - \mathbf{w}^*)$:

$$R(\mathbf{w}) = R(\mathbf{w}^*) + \frac{1}{2} \Delta \mathbf{w}^T \mathbf{H}(\mathbf{w}^*) \Delta \mathbf{w} + O\left((\Delta \mathbf{w})^3\right)$$

- To delete the weight w_i we have to ensure:

$$\mathbf{e}_i^T \Delta \mathbf{w} + w_i = 0$$

8.7 Pruning: “Optimal Brain Damage” (OBD)

- The error increase is determined by:

$$\sum_i H_{ii} (\Delta w_i)^2$$

- If we want to remove a weight then this term should be as small as possible. At the minimum the Hessian is positive definite, which means that for all $\mathbf{v}$

$$\mathbf{v}^T \mathbf{H} \mathbf{v} \geq 0 .$$

- Therefore: $\mathbf{e}_i^T \mathbf{H} \mathbf{e}_i = H_{ii} \geq 0$
- Therefore we choose $\Delta \mathbf{w} = - w_i \mathbf{e}_i$.
- Minimal error increase: $\min_i H_{ii} w_i^2$
- Removed weight: w_k with $k = \arg \min_i H_{ii} w_i^2$

8.7 Pruning: “Optimal Brain Surgeon” (OBS)

- The Taylor expansion and the constraint of removing weight i can be stated as a quadratic optimization problem

$$\begin{aligned} \min_{\Delta \mathbf{w}} \quad & \frac{1}{2} \Delta \mathbf{w}^T \mathbf{H} \Delta \mathbf{w} \\ \text{s.t.} \quad & \mathbf{e}_i^T \Delta \mathbf{w} + w_i = 0 \end{aligned}$$

- The **Lagrangian** is

$$L = \frac{1}{2} \Delta \mathbf{w}^T \mathbf{H} \Delta \mathbf{w} + \alpha (\mathbf{e}_i^T \Delta \mathbf{w} + w_i).$$

- The derivative has to be zero

$$\frac{\partial L}{\partial \Delta \mathbf{w}} = \mathbf{H} \Delta \mathbf{w} + \alpha \mathbf{e}_i = \mathbf{0}.$$

- So:

$$\Delta \mathbf{w} = -\alpha \mathbf{H}^{-1} \mathbf{e}_i$$

8.7 Pruning: “Optimal Brain Surgeon” (OBS)

Further we have

$$w_i = - e_i^T \Delta \mathbf{w} = \alpha e_i^T \mathbf{H}^{-1} e_i = \alpha \mathbf{H}_{ii}^{-1},$$

which gives

$$\alpha = \frac{w_i}{\mathbf{H}_{ii}^{-1}}$$

and results in

$$\Delta \mathbf{w} = - \frac{w_i}{\mathbf{H}_{ii}^{-1}} \mathbf{H}^{-1} e_i .$$

The objective becomes (using the above expression):

$$\frac{1}{2} \Delta \mathbf{w}^T \mathbf{H} \Delta \mathbf{w} = \frac{1}{2} \frac{w_i^2}{\mathbf{H}_{ii}^{-1}} .$$

8.7 Pruning: “Optimal Brain Surgeon” (OBS)

The criterion for selecting a weight to remove is the

$$\frac{1}{2} \frac{w_i^2}{H_{ii}^{-1}}$$

and the correction is done by

$$\Delta \mathbf{w} = - \frac{w_i}{H_{ii}^{-1}} \mathbf{H}^{-1} \mathbf{e}_i .$$

8.7 Pruning: “Optimal Brain Surgeon” (OBS)

Heuristics to improve OBS:

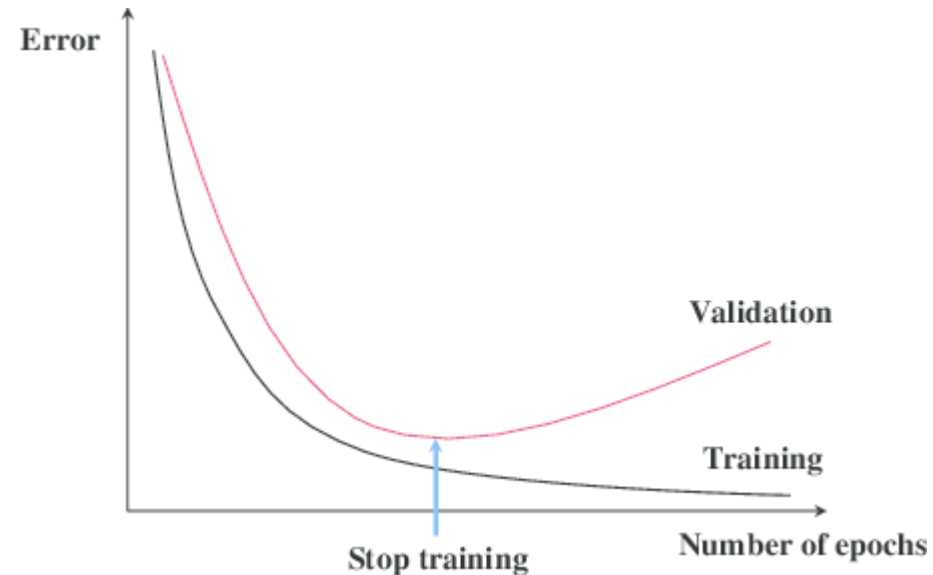
- Checking for the first order derivative $\nabla_w R(\mathbf{w}^*) = 0$
- Retraining after weight deletion
- Checking for $\|I - H^{-1} H\| > \beta$ to see the quality of the approximation of the inverse Hessian
- Check for weight changes larger than γ (e.g. $\gamma = 0.5$)

8.8 Early Stopping

Main idea:

Stop learning before the minimum is reached

- During learning the network will first extract the most dominant rules, that is the most important structures in the data.
- Later in the learning procedure characteristics are found which apply only to a few or just one example.



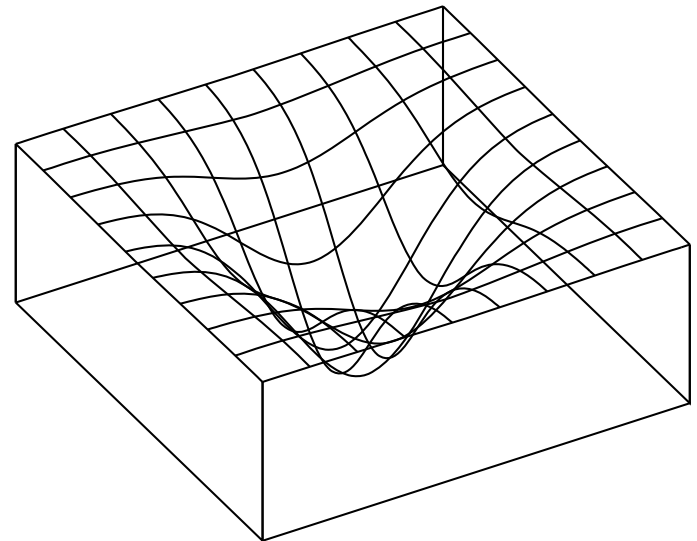
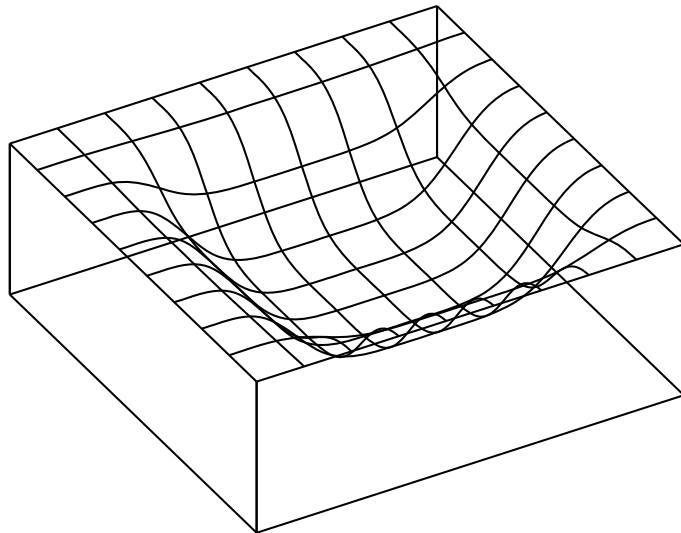
Drawbacks:

- Variability of early-stopping parameter is often high
- Strategy for using the validation set needed

8.9 Flat Minimum Search

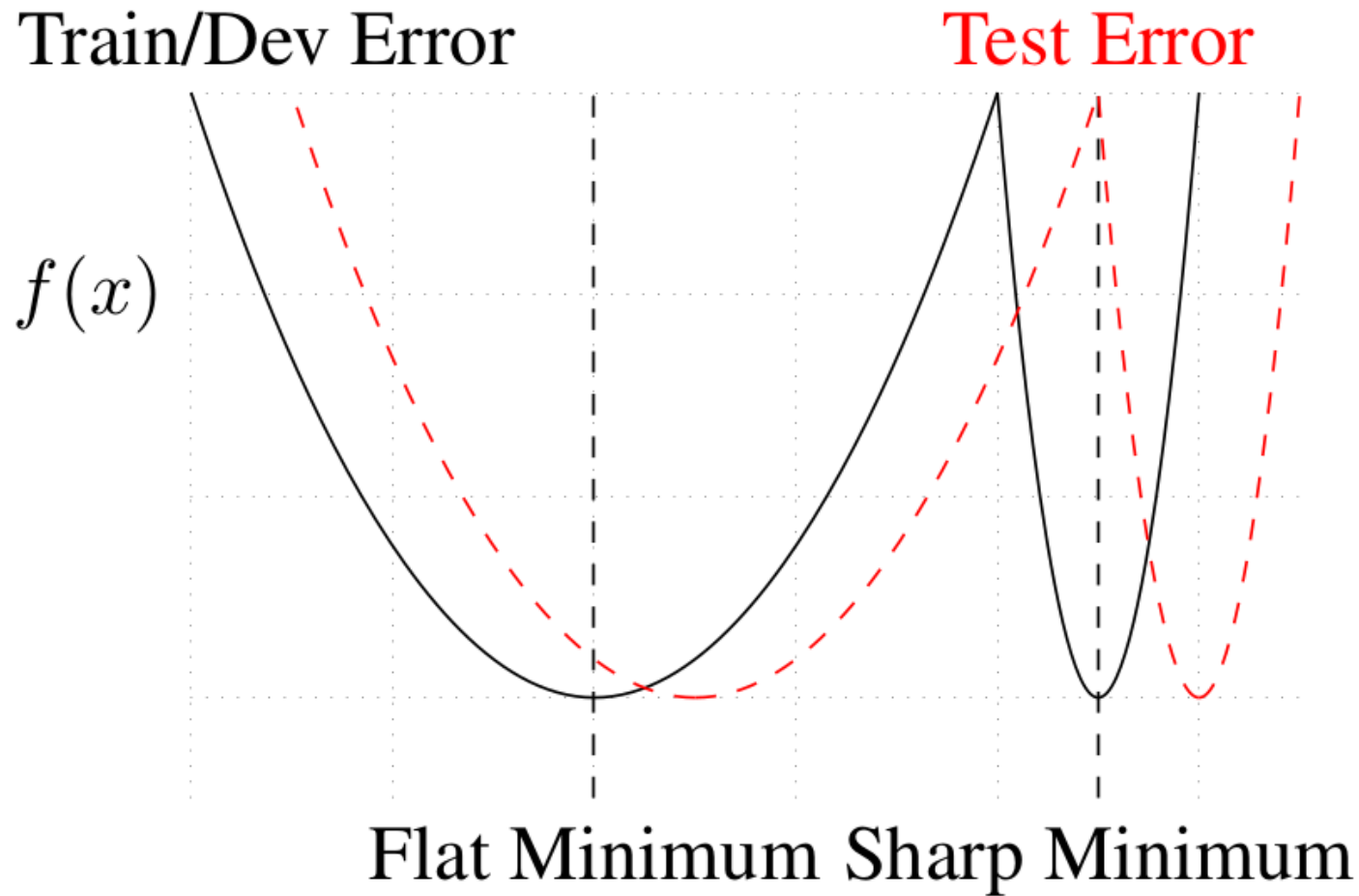
Main idea:

FMS searches for large regions in the weight space where the network function does not change but the empirical risk is small.



Hochreiter, S., & Schmidhuber, J. (1997). Flat minima. *Neural Computation*, 9(1), 1-42.

8.9 Flat Minimum Search



8.9 Flat Minimum Search

Flat minimum search adds an additional error term $\Omega(\mathbf{w})$ to the objective function $R(\mathbf{w})$. This error term $\Omega(\mathbf{w})$ describes the local flatness and has to be minimized.

$$\Omega(\mathbf{w}) = \frac{1}{2} \left(-W \log \epsilon + \sum_{i,j,l} \log \sum_{k=1}^K \left(\frac{\partial a_k}{\partial w_{ij}^{[l]}} \right)^2 + W \log \sum_{k=1}^K \left(\sum_{i,j,l} \frac{\left| \frac{\partial a_k}{\partial w_{ij}^{[l]}} \right|}{\sqrt{\sum_{k=1}^K \left(\frac{\partial a_k}{\partial w_{ij}^{[l]}} \right)^2}} \right)^2 \right)$$

$$\frac{\partial \Omega(\mathbf{w})}{\partial w_{uv}} = \sum_{k=1}^K \sum_{i,j,l} \frac{\partial \Omega(\mathbf{w})}{\partial \left(\frac{\partial a_k}{\partial w_{ij}^{[l]}} \right)} \frac{\partial^2 a_k}{\partial w_{ij}^{[l]} \partial w_{uv}}$$

$$\nabla_{\mathbf{w}} \Omega(\mathbf{w}) = \sum_{k=1}^K \mathbf{H}^k \left(\nabla_{\frac{\partial a_k}{\partial w_{ij}^{[l]}}} \Omega(\mathbf{w}) \right)$$

- $\mathbf{H}^k$ is the Hessian of output unit k . The complexity is $\mathcal{O}(K^2 W)$

8.10 Data Augmentation

- Main idea: Increase the size of the training set
- Empirical error without data augmentation:

$$R_{\text{emp}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N L(y^n, g(\mathbf{x}^n; \mathbf{w}))$$

- Empirical error with data augmentation:

$$R_{\text{reg}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N L(y^n, g(\mathbf{x}^n; \mathbf{w})) + \underbrace{\frac{1}{L} \sum_{l=1}^L L(y^l, g(\mathbf{x}^l; \mathbf{w}))}_{\text{error on augmented data}}$$

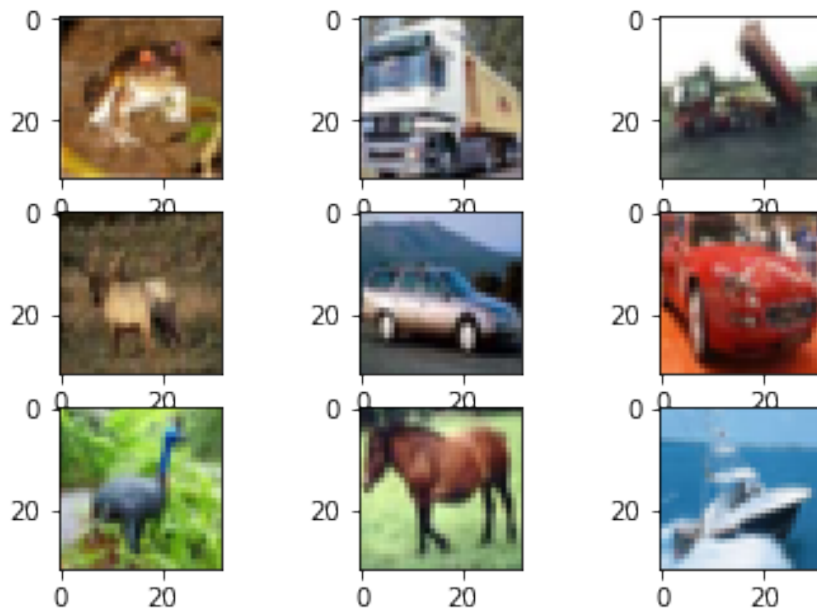
8.10 Data Augmentation

Strategies for image recognition tasks:

- Rotation
- Translation
- Cropping
- Brightness noise
- Shearing

8.10 Data Augmentation

Original

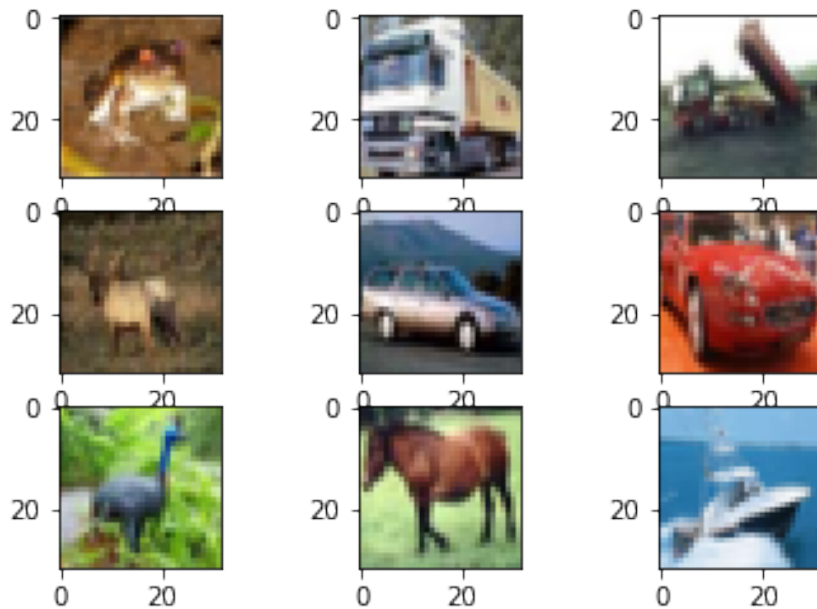


Rotation

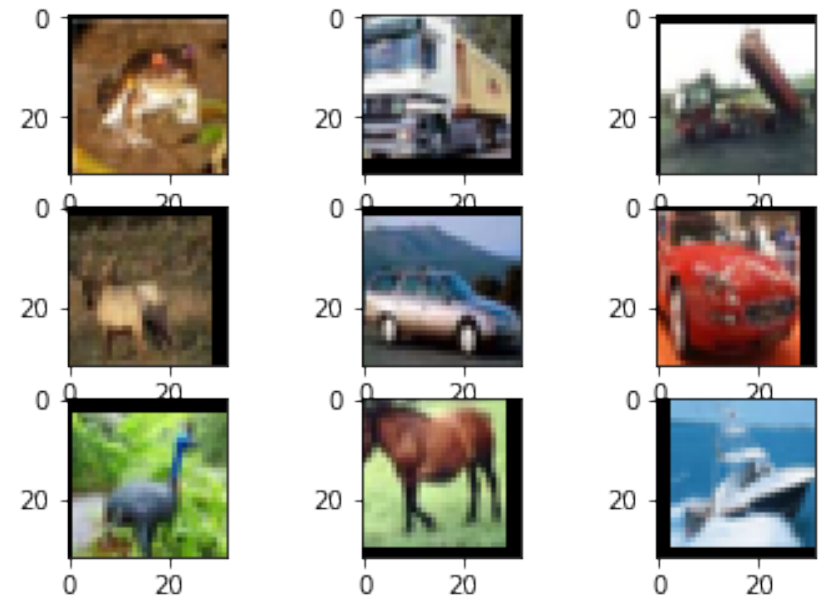


8.10 Data Augmentation

Original

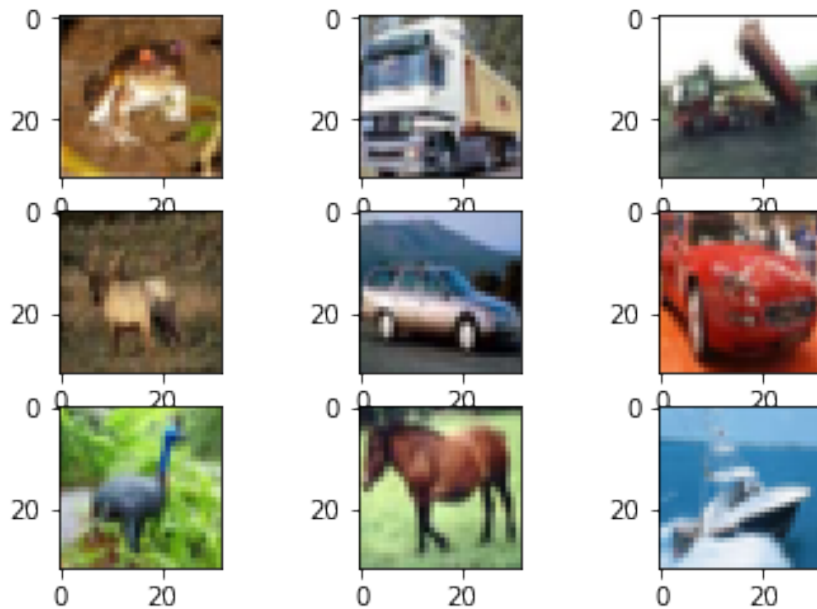


Translation



8.10 Data Augmentation

Original

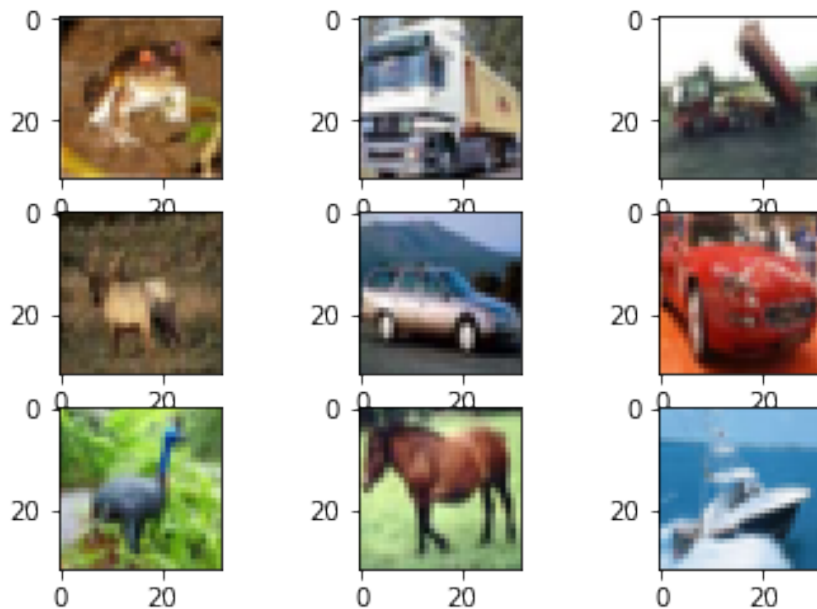


Cropping

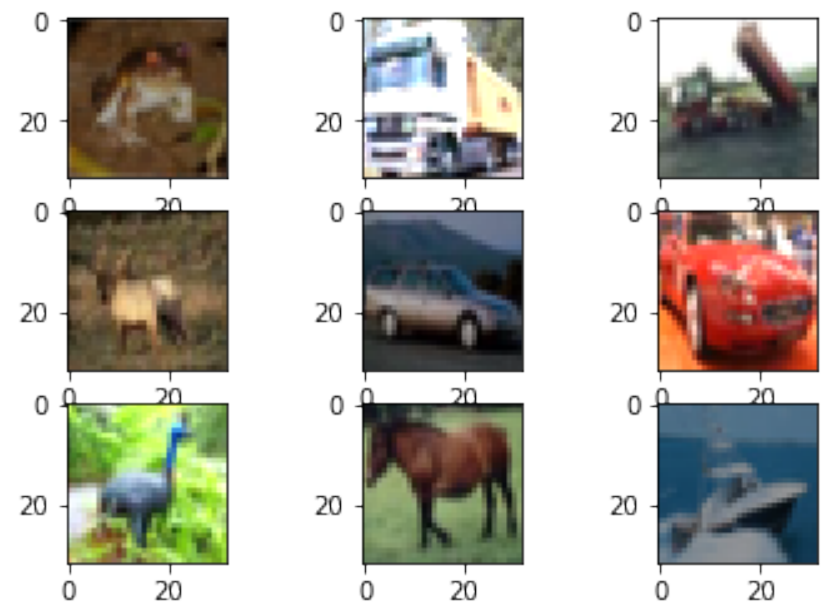


8.10 Data Augmentation

Original

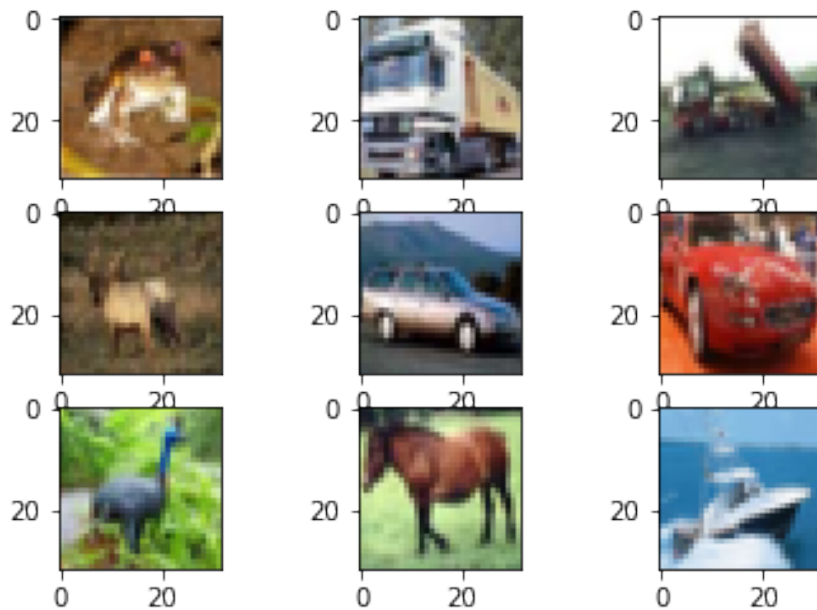


Brightness noise

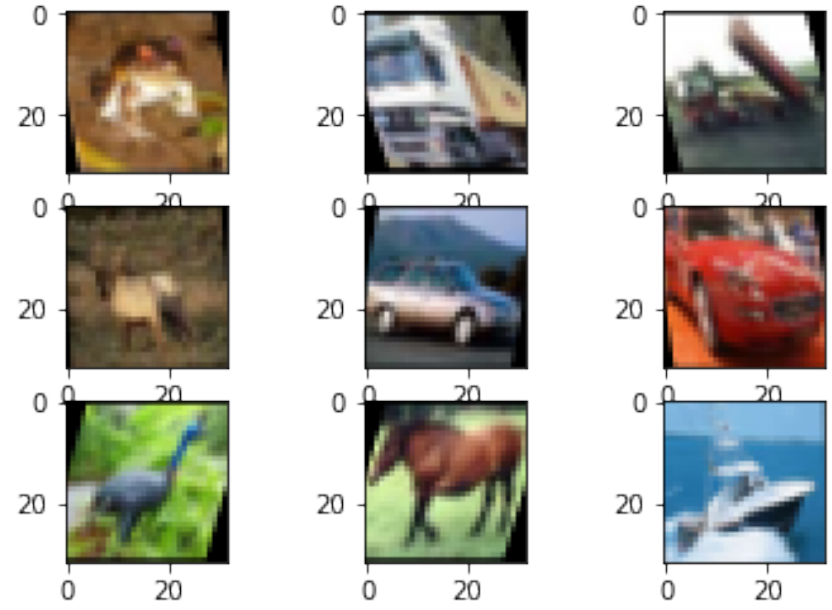


8.10 Data Augmentation

Original



Shearing



8.11 Self-regularization by SGD

- SGD minimizes an average potential over the posterior distribution of weights along with an entropic regularization term
- The so called Fokker-Planck equations describe the evolution of the system and can be used to provide an expression of the steady-state-distribution ρ^{ss} .
- This steady-state distribution over the weights estimated by the SGD is given by:

$$\rho^{\text{ss}} = \operatorname{argmin}_{\rho} \mathbb{E}_{\mathbf{w} \sim \rho} (\Phi(\mathbf{w})) - \frac{\eta}{2b} H(\rho)$$

H : Entropy of the system

b : Batch size

$\Phi(\mathbf{w})$: Original empirical error function

η : Learning rate

8.11 Self-regularization by SGD

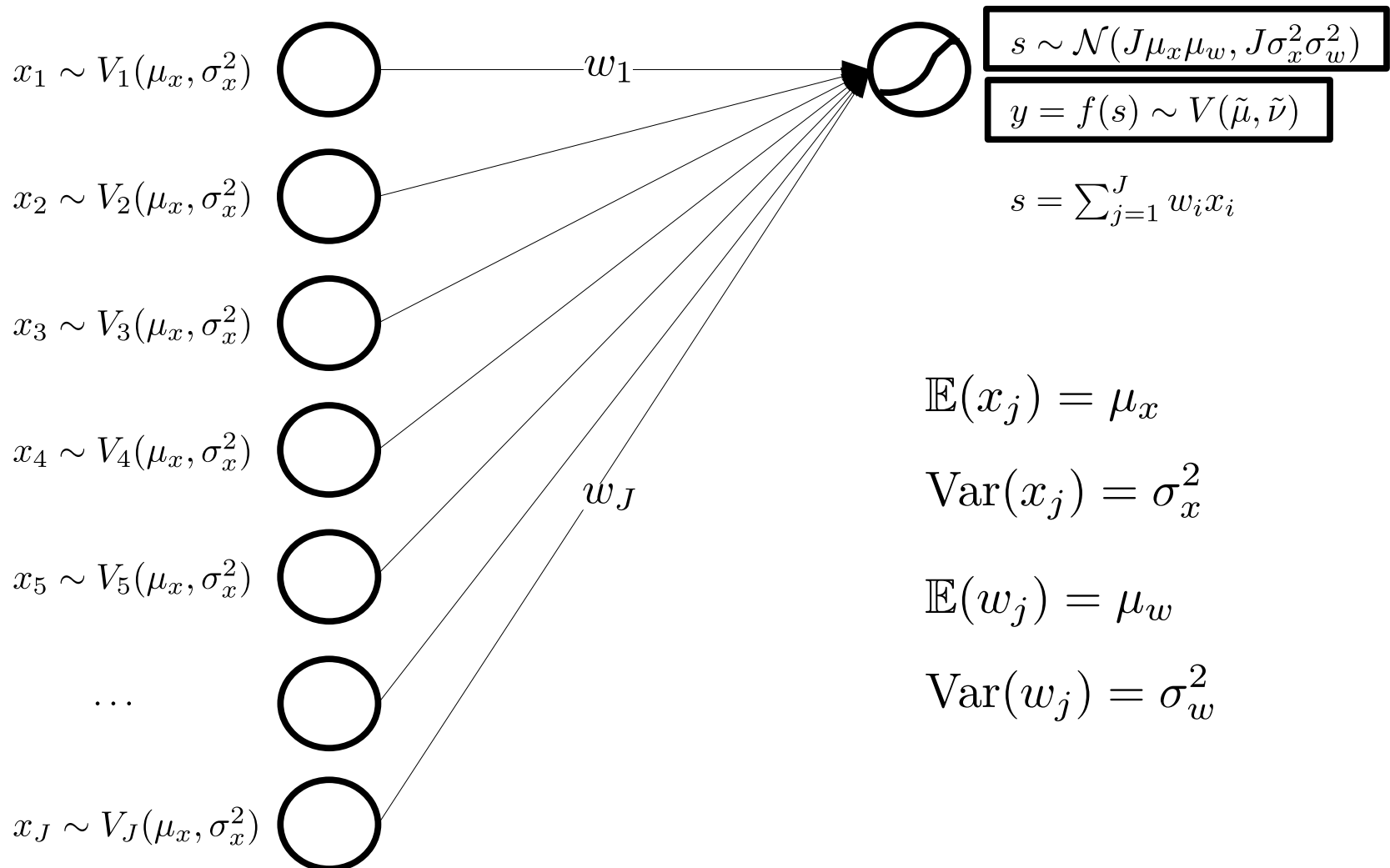
Consequences:

- SGD does not find minima of the original objective function.
- The deviation scales linearly with $\beta^{-1} = \eta/(2b)$.
- Large learning rates and small batch sizes have a regularization effect on learning since $\beta^{-1} = \frac{\eta}{2b}$.
- The learning rate should linearly scale with batch size.

Chaudhari, P., & Soatto, S. (2018, February). Stochastic gradient descent performs variational inference, converges to limit cycles for deep networks. In 2018 Information Theory and Applications Workshop (ITA) (pp. 1-10). IEEE.

9. Initialization

9.2 Signal propagation (SP) theory: Central limit theorem (CLT)



9.2 Variance propagation: LeCun's initialization (LeCun, 1998)

- Already suggested in 1998 to initialize as follows:

$$W_{ij} \sim \mathcal{N}(0, \frac{1}{J})$$

$$W_{ij} \sim \mathcal{U}(-\sqrt{\frac{3}{J}}, \sqrt{\frac{3}{J}}).$$

9.2 SP theory for initialization: trade-off between forward and backw.

- Glorot's initialization (Glorot and Bengio, 2010):

$$W_{ij} \sim \mathcal{N}(0, \frac{2}{J+I})$$

$$W_{ij} \sim \mathcal{U}(-\sqrt{\frac{6}{J+I}}, \sqrt{\frac{6}{J+I}})$$

9.2 SP theory for initialization: trade-off between forward and backw.

- Glorot's initialization (Glorot and Bengio, 2010):

$$W_{ij} \sim \mathcal{N}(0, \frac{2}{J + I})$$

$$W_{ij} \sim \mathcal{U}(-\sqrt{\frac{6}{J + I}}, \sqrt{\frac{6}{J + I}})$$

9.3 Non-linearities: He's initialization (2015)

- Account for the effect of ReLU:

$$J\sigma_w^2 \frac{1}{2} = 1$$

Initialize weights that counter keep unit variance:

$$\sigma_w^2 = \frac{2}{J}.$$

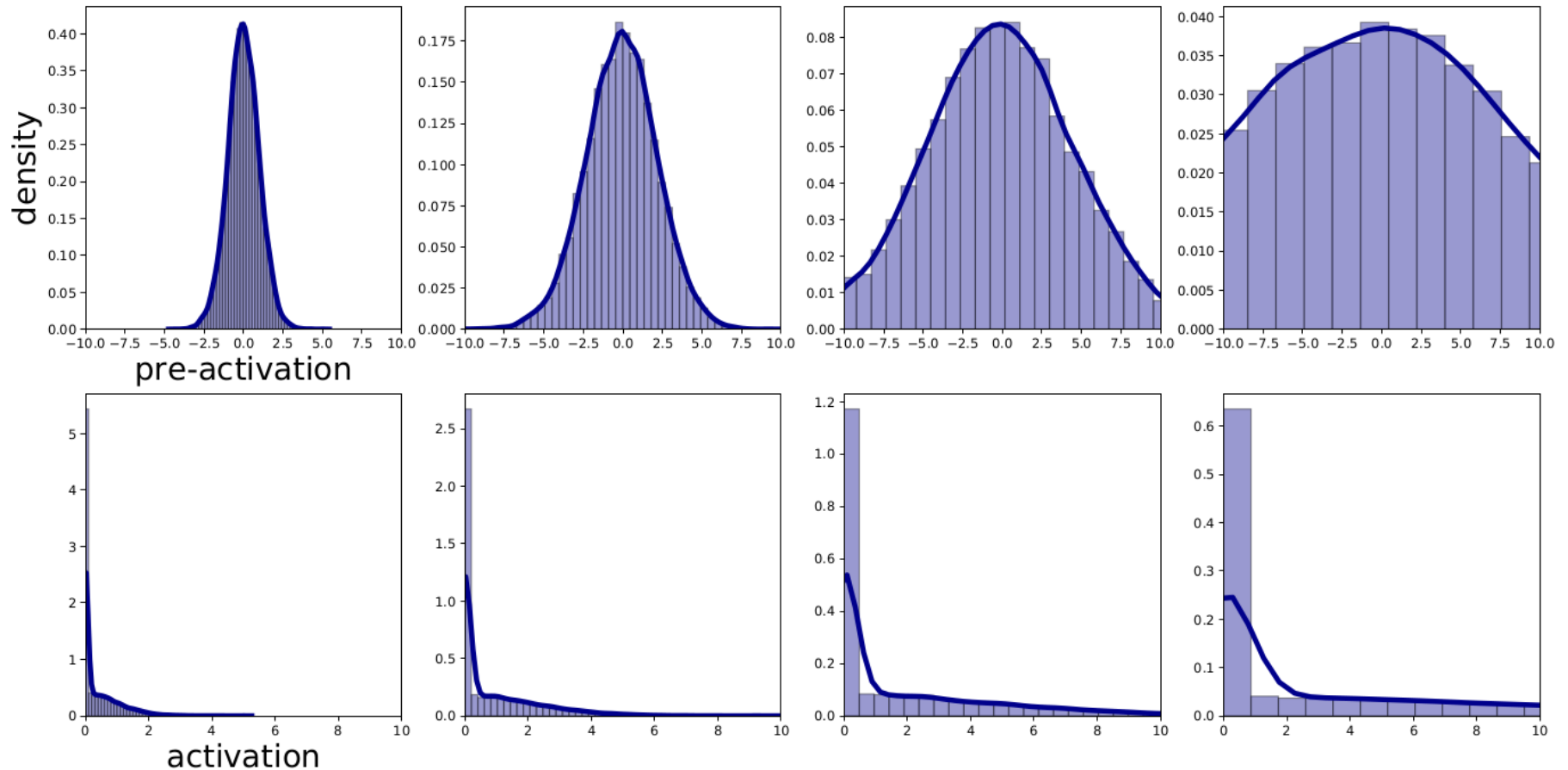
- Resulting initialization:

$$W_{ij} \sim \mathcal{N}(0, \frac{2}{J})$$

$$W_{ij} \sim \mathcal{U}(-\sqrt{\frac{6}{J}}, \sqrt{\frac{6}{J}})$$

10. Normalization

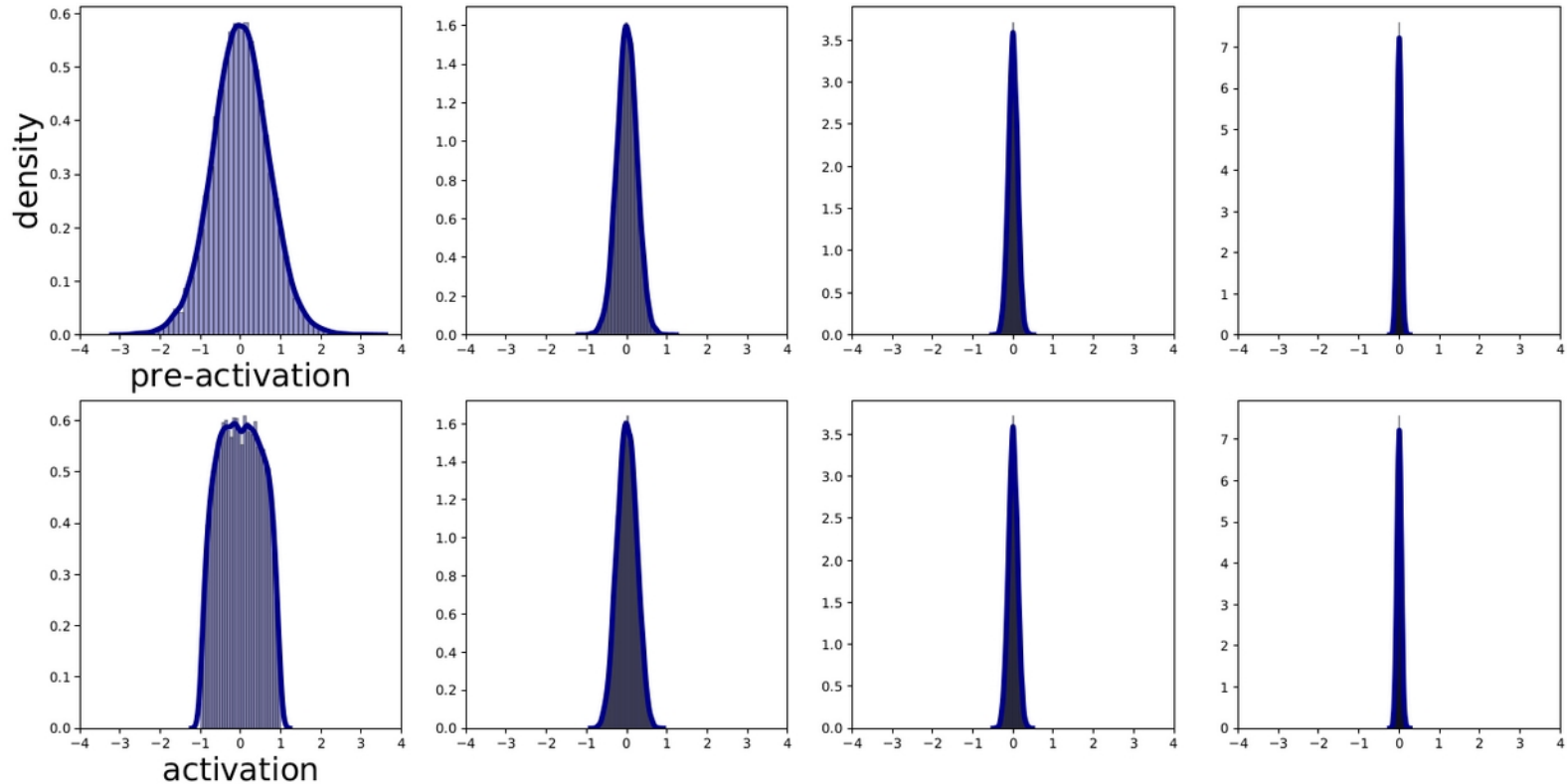
Vanishing and exploding activations



Activation function: ReLU

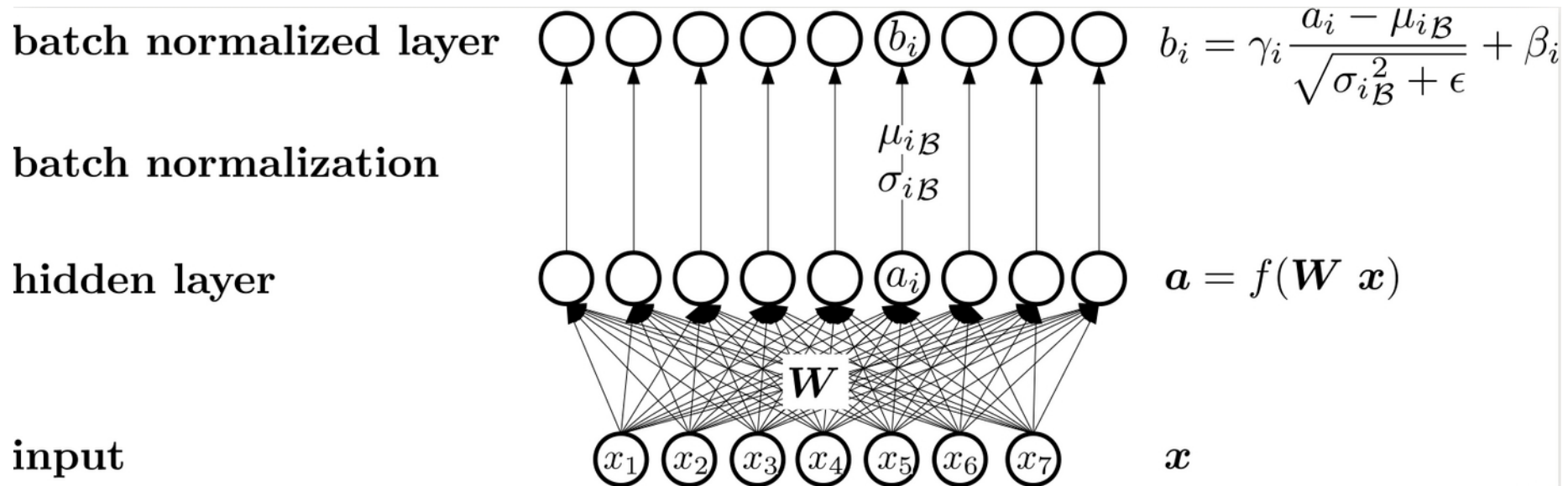
10. Normalization

Vanishing and exploding activations



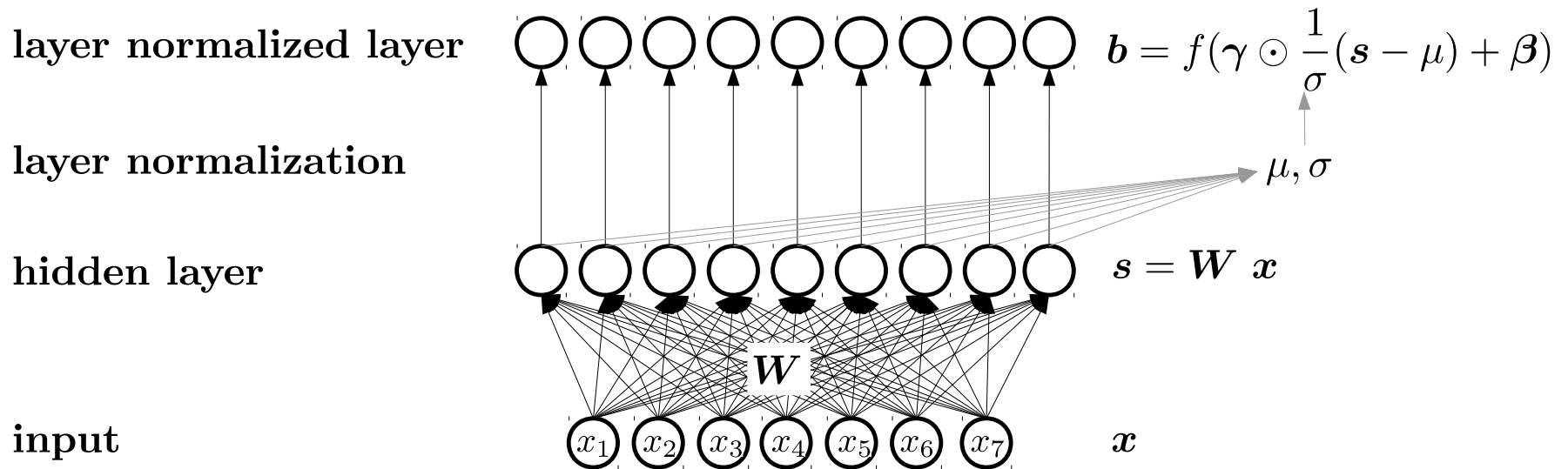
Activation function: Tanh

10.1. Batch normalization



- Each neuron “learns” a mean μ_i and a variance σ_i^2 for a batch
- The operation is differentiable wrt inputs

10.2. Layer normalization



10.3. Weight normalization

- We further assume that there is a loss function $L(\mathbf{y}, g(\mathbf{x}, \mathbf{w}))$ which depends on $\mathbf{w}$, such that we can have a gradient with respect to the parameters $\mathbf{w}$: $\nabla_{\mathbf{w}} L$, then:

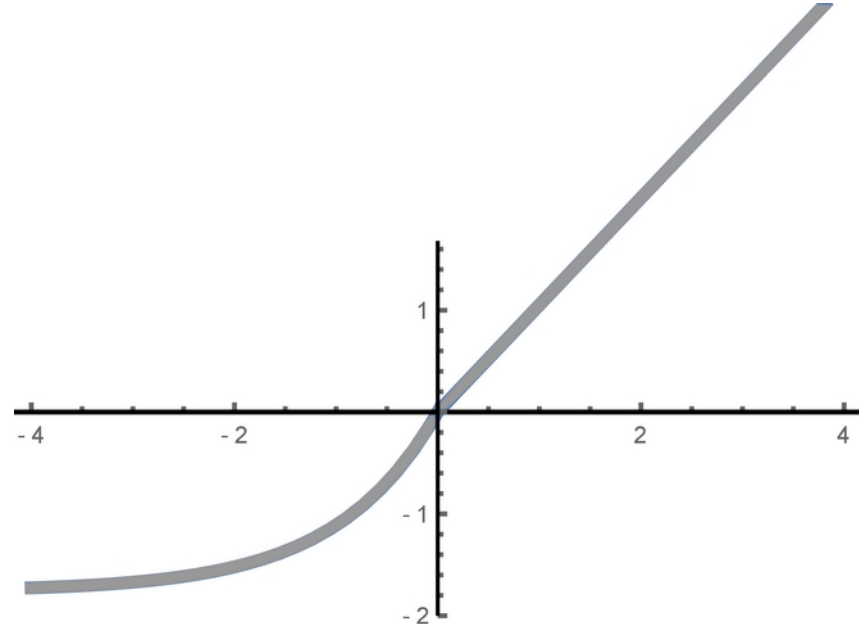
$$\nabla_g L = \frac{(\nabla_{\mathbf{w}} L) \cdot \mathbf{v}}{\|\mathbf{v}\|}$$
$$\nabla_{v_i} L = \frac{g}{\|\mathbf{v}\|} \left(1 - \frac{v_i^2}{\|\mathbf{v}\|^2} \right) \nabla_{w_i} L,$$

where $\nabla_{\mathbf{w}} L$ is a row vector and we have used the quotient rule for the second term.

10.4. Self-normalization: Scaled exponential linear unit (SELU)

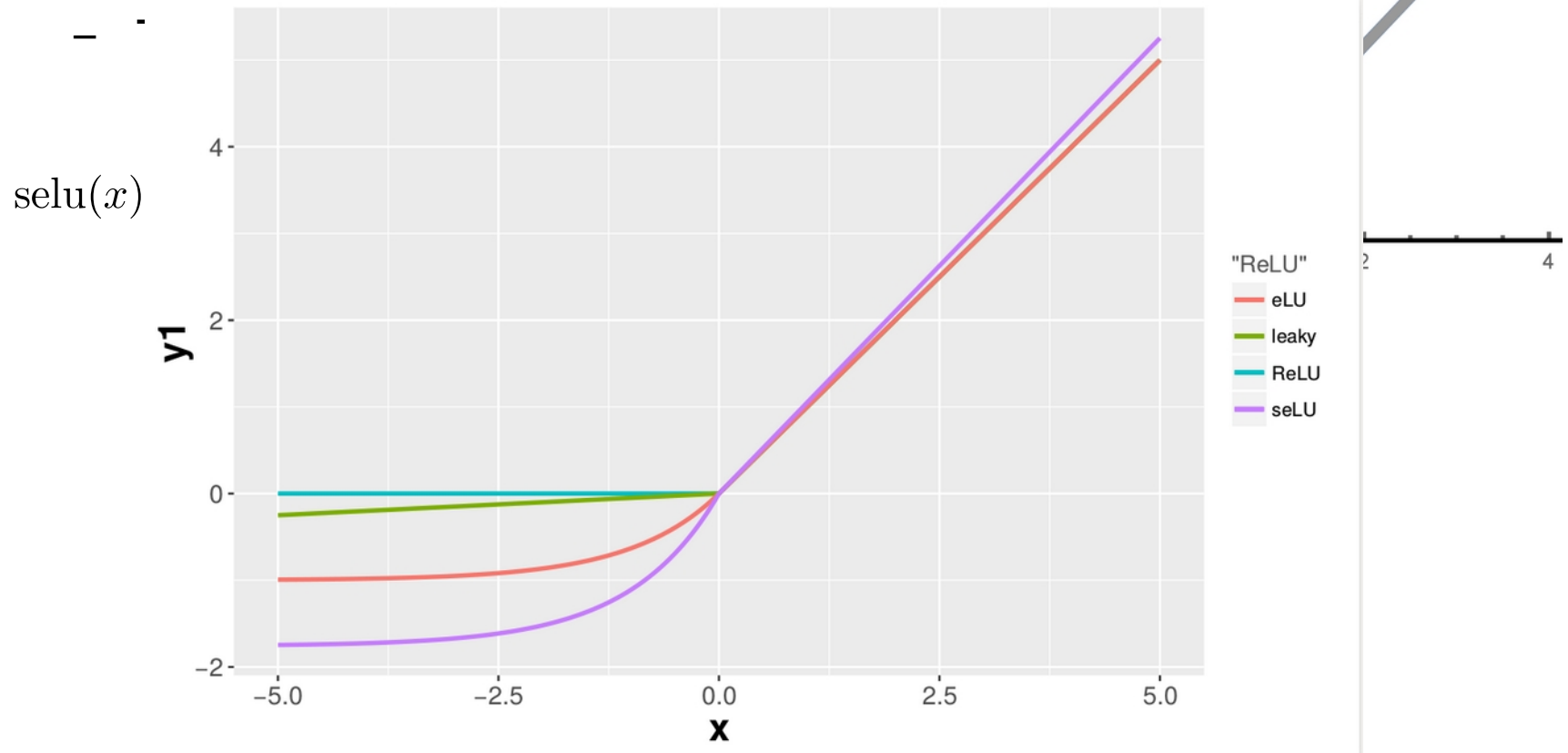
- SELU:
 - Two adjustable parameters

$$\text{SELU}(x) = \lambda \begin{cases} x & \text{if } x > 0 \\ \alpha e^x - \alpha & \text{if } x \leq 0 \end{cases}.$$



10.4. Self-normalization: Scaled exponential linear unit (SELU)

- SELU:



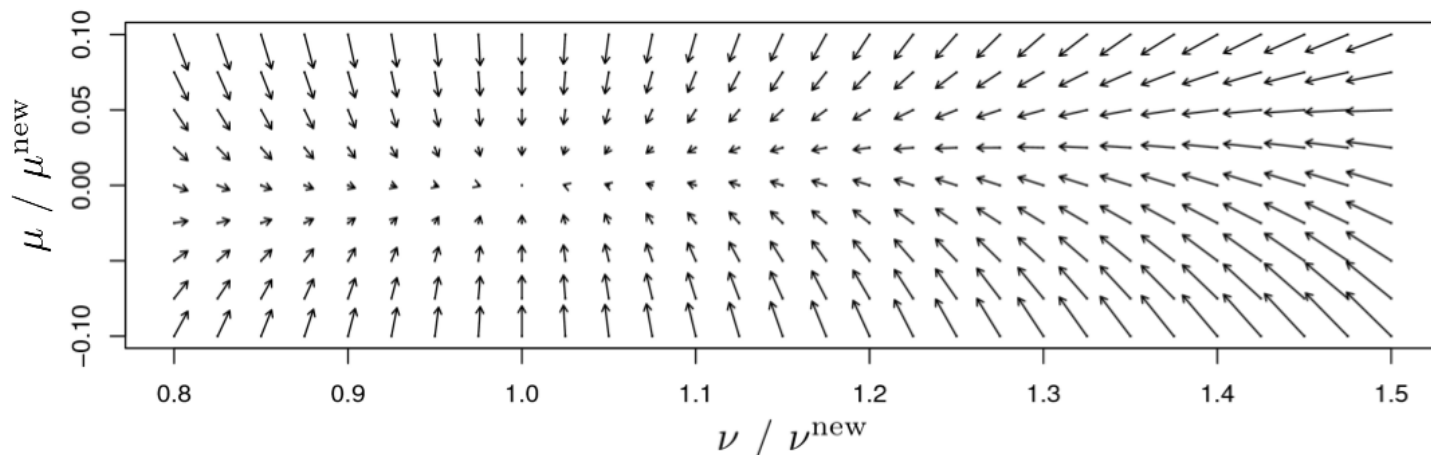
10.4. Self-normalization:

Stable fixed point at zero mean unit var

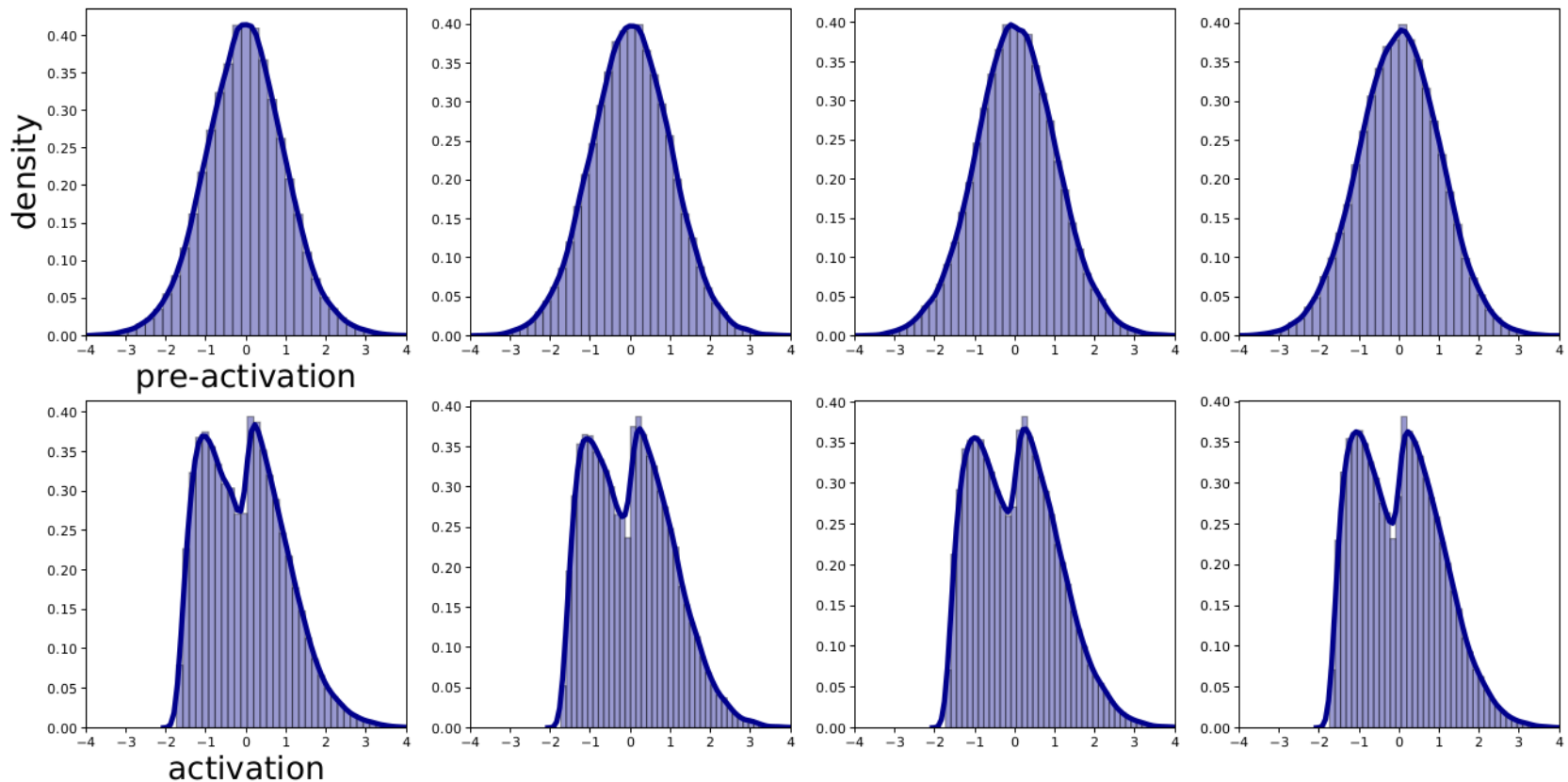
- Jacobian of mapping g at fixed point $(0,1)$:

$$\mathcal{J}(\mu, \nu) = \begin{pmatrix} \frac{\partial \mu(\mu, \nu)}{\partial \mu} & \frac{\partial \mu(\mu, \nu)}{\partial \nu} \\ \frac{\partial \nu(\mu, \nu)}{\partial \mu} & \frac{\partial \nu(\mu, \nu)}{\partial \nu} \end{pmatrix}, \quad \mathcal{J}(0, 1) = \begin{pmatrix} 0.0 & 0.088834 \\ 0.0 & 0.782648 \end{pmatrix}.$$

- spectral norm of Jacobian is smaller than one
- mapping g is a contraction mapping
- $(0,1)$ is a stable fixed point



10.4. Self-normalization: “Vanishing and exploding activations”



Summary

- Fully-connected networks (FCN)
 - Forward pass, backward pass, VGP
 - Training
- Convolutional neural networks (CNNs)
 - Convolutional operation; other operations
 - Conv blocks
 - Data dimensions; number of params
- Regularization
- Initialization
- Normalization